Roith *et al. Visualization in Engineering* (2017) 5:20
DOI 10.1186/s40327-017-0057-y

Visualization in Engineering

RESEARCH Open Access

Supporting the building design process with graph-based methods using centrally coordinated federated databases

Johannes Roith[1], Christoph Langenhan[2*] and Frank Petzold[2]

Abstract

Background: Technological developments and globalized working processes have transformed the building process. However, current digital semantic building models no longer adequately represent the increasing complexity of modern building projects. The potential of combining agent-based and graph-based methods, for example for energy calculations or spatial research strategies, is not fully exploited.

Methods: In our system, users search for building floorplans, for example as sources of inspiration, by creating conceptual hand-drawn sketches of building parts on multi-touch devices. The sketch is analyzed and used to query a federated database system comprising a building information model server, the graph database neo4j and the content management system mediaTUM. Users interact with client applications that show and continuously update a list of floorplans by sending queries to a central coordinator service. In the paper, we describe the coordinator that enables our databases to appear as a single smart information system to search for digital information about buildings and visualize their floorplans. The application case comprises search by drawing the initial design idea of a building to find similar floorplans e.g. as source of inspiration.

Results: Our federated database system is queried using semantic building floorplan fingerprints, which are formalized as graphs and encoded in a common schema, like our AGraphML, to represent spatial configurations and perform graph matching. As graph matching is computationally expensive, the coordinator needs to analyze the queries, separate different fingerprints and metadata and pass it to specialized agents, implemented as microservices, for processing. The returned result sets are combined and the results are visualized. The use of multiple agents facilitates the recombination of the data from the underlying disparate data sources and also serves to support different execution strategies which are chosen using properties of the query and a set of predefined rules.

Conclusions: To deal with complex and dynamic queries, as well as dynamic results that are updated while some agents are still executing, a caching framework was developed which takes the similarity measures of the graph-based building representation into account when determining query equality.

Keywords: Research strategies, Building information modelling, Federated databases

Background

As the complexity of building tasks and requirements increases, designers often find themselves confronted with interdisciplinary problems that go beyond the specific challenges and methods of architecture and engineering. The iterative nature of the design process results in a continuous exchange between creative, analytical and evaluative activities, through which the designer explores and identifies promising design variants. The ability to compare and evaluate relevant reference examples of already built or designed buildings helps designers assess their own design explorations and informs the design process. Most computational search methods available today rely on textual rather than graphical approaches to represent information. However, textual descriptions are not sufficient to adequately describe spatial configurations

*Correspondence: langenhan@tum.de
[2]Department of Architecture, Technical University of Munich, Munich, Germany
Full list of author information is available at the end of the article

Springer Open

© The Author(s). 2017 **Open Access** This article is distributed under the terms of the Creative Commons Attribution 4.0 International License (http://creativecommons.org/licenses/by/4.0/), which permits unrestricted use, distribution, and reproduction in any medium, provided you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license, and indicate if changes were made.

Roith *et al. Visualization in Engineering* (2017) 5:20
DOI 10.1186/s40327-017-0057-y

Visualization in Engineering

RESEARCH Open Access

利用集中协调的联邦数据库，通过基于图的方法支持建筑设计过程

约翰内斯·罗伊特[1]，克里斯托夫·朗根汉[2]*和弗兰克·佩茨奥尔德[2]

摘要

背景：技术进步与全球化的工作流程已深刻改变了建筑建造过程。然而，现有的数字化语义建筑模型已难以充分表征现代建筑项目的日益复杂性。例如，在能源计算或空间研究策略等领域，基于智能体与基于图的方法相结合的潜力尚未得到充分挖掘。

方法：在我们的系统中，用户通过在多点触控设备上绘制建筑各部分的概念性手绘草图，来搜索建筑平面图，例如作为设计灵感的来源。该草图经分析后用于查询一个联邦数据库系统，该系统由建筑信息模型服务器、图数据库Neo4j以及内容管理系统mediaTUM组成。用户通过客户端应用程序与系统交互，这些应用程序会向中央协调服务发送查询请求，并实时显示和持续更新平面图列表。在本文中，我们详细描述了该协调服务，它使多个数据库能够作为一个统一的智能信息系统，用于检索建筑的数字化信息并可视化其平面图。本应用案例包括通过绘制建筑的初始设计构思来搜索相似的平面图，例如作为设计灵感的参考。

结果：我们的联邦数据库系统通过语义化的建筑楼层平面指纹进行查询，这些指纹被形式化为图结构，并以统一的模式（如我们的AGraphML）进行编码，用于表示空间配置并执行图匹配。由于图匹配的计算开销较高，协调器需要对查询进行分析，分离不同的指纹与元数据，并将其传递给作为微服务实现的专用代理进行处理。返回的结果集经过合并后予以可视化展示。采用多代理机制不仅有助于整合来自底层异构数据源的数据，还能够支持基于查询属性及一组预定义规则选择的不同执行策略。

结论：为应对复杂且动态的查询，以及在部分代理仍在执行过程中即被更新的动态结果，我们开发了一种缓存框架，在判定查询等价性时考虑了基于图的建筑表示的相似性度量。

关键词：研究策略，建筑信息建模，联邦数据库

背景

随着建筑任务与需求复杂性的不断提升，设计师往往面临跨学科问题，这些问题超出了建筑学与工程学各自特定的挑战与方法范畴。设计过程的迭代特性导致创造性活动、分析性活动与评价性活动之间持续交互，借此设计师得以探索并识别具有前景的设计方案。比较与评估已建成或已设计建筑的相关参考实例的能力，有助于设计师对其自身的设计探索进行评估，并为设计过程提供指导。然而，当前大多数计算搜索方法仍依赖于文本而非图形化的信息表达方式；但仅凭文本描述尚不足以充分刻画空间构型。

*Correspondence: langenhan@tum.de
[2]Department of Architecture, Technical University of Munich, Munich, Germany
Full list of author information is available at the end of the article

Springer Open

© The Author(s). 2017 **Open Access** This article is distributed under the terms of the Creative Commons Attribution 4.0 International License (http://creativecommons.org/licenses/by/4.0/), which permits unrestricted use, distribution, and reproduction in any medium, provided you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license, and indicate if changes were made.

such as floor plans. To address these shortcomings, Langenhan et al. (2013) introduced a novel approach which facilitates the automatic lookup of reference solutions from a repository using graphical search keys. For the search key, the notion of a building floorplan fingerprint was introduced which describes the main characteristics of a building's design. This forms the basis for assessing the similarity of different reference solutions to a specified problem and serves, accordingly, as an index for the building model repository.

This paper describes an information retrieval system that allows users to search for building models that are similar to a hand-drawn sketch of a building or parts of a building on a multi-touch device to support the design process of architects. The design process is an iterative process (Buxton 2007) of searching for a plausible solution involving a continual back and forth in which potential solutions are developed by various means before narrowing down the selection to the most promising candidates. Architects typically work with traditional design tools in the early design stages, such as model-making, sketches and the use of reference examples. Using reference examples is an acknowledged method in both architectural education and architectural practice and helps in learning design principles and guiding the design process, as well as for inspiration or even as an explicit solution (Richter 2010). Digital equivalents have already been devised for several traditional design tools, however many of these do not fully translate the original strengths of the analogue methods into the digital world and fail to make full use of digital possibilities.

The approach discussed aims to overcome these shortcomings and proposes a new paradigm for designing with reference examples that takes into account the typical way a designer works in the early design phases and makes full use of current methods and technologies. Finding relevant reference examples with traditional technologies is often tedious, and at present there is a lack of appropriate IT support for this task. A central problem is finding a way of determining similarity between the designer's query and the data stock. The solution we are proposing (Langenhan et al. 2013) is based on graph representations that capture the topological relationships between spaces. For indexing and determining similarities, the use of semantic fingerprints (Langenhan and Petzold 2010) has been proposed as a way of describing and visualizing the arrangement of building floorplans in a manner analogous to the way that fingerprints can be used to identify a person. The system derives semantic fingerprints 'representing e.g. accessibility and adjacency as features for search criteria' from a reference solution that describes the spatial relationship of rooms extracted from building information models. In the application area described in this paper (see Fig. 1), the system assists the designer by analyzing sketches made

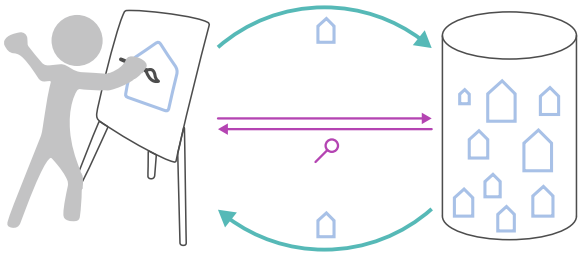


Fig. 1 An application scenario for knowledge-based design

in the early design phases and deriving a structure that can be compared with the fingerprints in the information repository. Using a graphical user interface, the designer can sketch spatial configurations as a bubble diagram, freehand sketch or schematic digital layout to visually explore the spatial configuration.

Floorplans are a way of representing the structure of a spatial configuration visually, which can be formalized mathematically in graphs. In our system, each floorplan sketch is first converted to a graph. Rooms are represented as nodes and doors/walls are represented as edges. The graph is stored as AGraphML, which is based on the GraphML standard (GraphML 2017). Next, *fingerprints* are computed which can be compared to a database of existing graphs. Users can define which fingerprints they want to use for matching or let the coordinating service select appropriate ones (Fig. 2).

There are six main reasons why we use coordinated microservices as our approach: First, we wish to support different types of fingerprints, each potentially implemented using its own microservice. Second, for each fingerprint, there may be multiple execution strategies available. This is particularly important for graph matching which often requires different strategies for different types of graphs in order to execute efficiently (Conte et al. 2004). It then becomes necessary to select the best service, e.g. based on a set of *rules*. Third, the building data is stored in a federated database system and it is useful to be able to access multiple databases in a single query. In our case the database system consists of the open source building information model server, the graph database neo4j, which stores the fingerprints, and the content management system mediaTUM, which holds pictures and metadata. Fourth, we want to be able to express complex queries that involve various microservices and obtain combined and ranked results. Fifth, since the computation of some fingerprints can be very expensive, it is necessary to reuse as much of the previous responses of the individual microservices as possible. Since reusability depends on the graph structure, an application-specific caching framework is needed. To address these problems while keeping the clients and microservices simple, we propose a central web service called the *Coordinator*

例如平面图。为解决这些不足，Langenhan 等人（2013）提出了一种新颖的方法，该方法借助图形化搜索键，实现了从知识库中自动检索参考解。在该方法中，引入了建筑平面图指纹的概念，用于描述建筑设计的主要特征。这一概念构成了评估不同参考解与特定问题之间相似性的基础，并相应地作为建筑模型知识库的索引。

本文介绍了一种信息检索系统，该系统允许用户在多点触控设备上搜索与建筑或其局部的手绘草图相似的建筑模型，以支持建筑师的设计过程。设计过程是一个迭代的过程（Buxton 2007），即通过不断反复地探索可行的解决方案，在此过程中，人们会采用多种方式生成潜在方案，然后逐步将选择范围缩小至最具前景的候选方案。在设计的早期阶段，建筑师通常使用传统的设计工具，如模型制作、手绘草图以及参考实例。在建筑教育与实践中，参考实例已被公认为一种行之有效的方法，它有助于学习设计原理、指导设计流程，并为设计提供灵感，甚至可以直接作为解决方案（Richter 2010）。针对若干传统设计工具，已有相应的数字化替代方案被开发出来，然而其中许多方案未能将模拟方法的固有优势完整地迁移到数字环境中，也未能充分发掘数字技术的潜力。

所讨论的方法旨在克服这些不足，并提出了一种基于参考实例的设计新范式，该范式充分考虑了设计师在设计初期的典型工作方式，并充分利用了当前的方法与技术。采用传统技术寻找相关参考实例往往耗时费力，且目前尚缺乏针对这一任务的适当信息技术支持。其中的核心问题在于如何确定设计师的查询与数据资源库之间的相似性。我们提出的解决方案（Langenhan 等，2013）基于图模型表示，以刻画空间之间的拓扑关系。为实现索引与相似性度量，已有研究提出利用语义指纹（Langenhan 和 Petzold，2010），通过类似于指纹识别个体的方式，对建筑平面布局进行描述与可视化。该系统从建筑信息模型中提取的、用于描述房间空间关系的参考方案中，衍生出表征诸如可达性与邻接关系等特征的语义指纹，作为检索依据。在本文所述的应用场景中（见图1），该系统通过分析设计师绘制的草图来提供辅助。

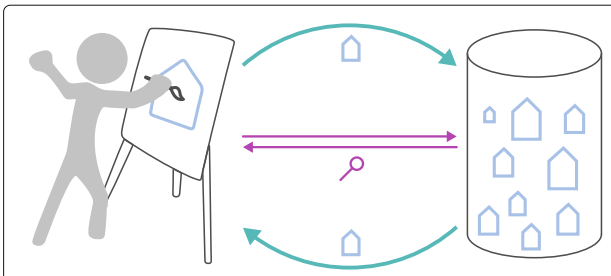


图1 基于知识的设计应用场景

在设计的早期阶段，构建一种可与信息库中的特征指纹进行比对的结构。借助图形用户界面，设计师可通过气泡图、手绘草图或示意性数字布局等方式勾勒空间构型，从而直观地探索空间布局。

平面图是一种以可视化方式表征空间构型结构的方法，其数学形式可进一步抽象为图模型。在本系统中，每幅平面草图首先被转换为一张图；其中，房间被表示为节点，门或墙体被表示为边。该图以AGraphML格式存储，其基础为GraphML标准（GraphML 2017）。随后，系统会计算指纹特征，并将其与现有图数据库中的指纹进行比对。用户可自行指定用于匹配的指纹类型，也可由协调服务自动选择合适的指纹（图2）。

我们采用协调式微服务架构的主要原因有六个：首先，我们希望支持多种类型的指纹，每种指纹均可由独立的微服务实现。其次，对于每种指纹，可能存在多种执行策略。这一点在图匹配任务中尤为重要，因为针对不同类型的图，往往需要采用不同的策略才能高效执行（Conte 等，2004）。因此，有必要根据一组规则选择最优的服务。第三，建筑数据存储于联邦数据库系统中，能够在单个查询中访问多个数据库十分有益。在我们的场景中，该数据库系统由开源建筑信息模型服务器、用于存储指纹的图数据库 Neo4j，以及用于存放图片和元数据的内容管理系统 mediaTUM 组成。第四，我们希望能够表达涉及多个微服务的复杂查询，并获得经过整合与排序的结果。第五，由于部分指纹的计算开销较大，有必要尽可能复用各微服务先前的响应结果。鉴于可复用性取决于图结构，因此需要一个面向特定应用的缓存框架。为在解决上述问题的同时保持客户端与微服务的简洁性，我们提出了一种名为“协调器”的中心化 Web 服务。

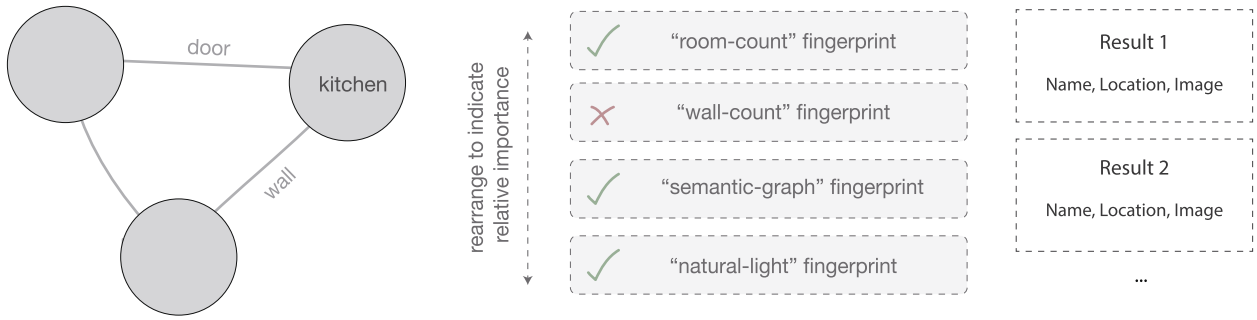


Fig. 2 The simplified graph (left) is extracted from a sketch and fingerprints are selected (right)

which processes queries and distributes their parts to the microservices. Sixth, the coordinator can precompute visualization data for the client applications.

Software architecture

To realize the semantic search engine, we propose a system of data integration[1] using a federated information system in which different autonomous sources of information (e.g. a BIM server, graph database and CMS) can be integrated using a common XML schema and queried as a single information source using a REST approach. This makes it possible to store and process information efficiently according to their specific properties. In contrast to other *data warehouse* approaches, a federated information system does not copy the various sources but rather queries the respective sources individually using processing components and bundles the results for further processing by the coordinated system of components in the client applications. For offline processing and extraction of building floorplan fingerprints, components are needed that a) analyze and augment unstructured data sources (CMS) and b) components that extend and attribute information to concepts in structured data sources (BIM). Figure 3 shows the software architecture of the information system which draws on the software architecture of the semantic search engine by (2012, p. 461). Topological information is extracted offline from formal (BIM) and informal (CMS) data sources using various methods and formalized in the form of graphs (fingerprints). The offline processing must structure the data in such a way that the user is presented with useful results within a reasonable time frame during online processing.

The components shown are groups of individual applications in the information cloud (servers) and on different end devices (clients). On the server-side the system is comprised of *data storage*, *processing* and *coordination* components. Possible clients include a desktop computer with mouse and keyboard (stationary workstation, e.g. in the office), tablet computers (transportable device, e.g. for client visits and meetings), smartphones (mobile, e.g. on-site use or site research) and a multi-touch table (stationary design environment).

The applications on the various end devices query the information cloud (Fig. 4). The information system for assisting the designer in the early design phases comprises a series of components for data storage and processing in order to store and process semantic building model data, graph data and text.

The application contexts and individual design processes differ from case to case. Different abstractions are therefore necessary to allow the designer to communicate his or her mental model as completely and accurately as possible. Because designers often only have a vague and incomplete notion of the design in the early design phases, the system needs to accommodate a high degree of freedom of expression and types of abstraction.

Different layers of abstraction

The system has to provide interaction approaches that allow the user to express a mental model that can be analyzed and compared with stored model schemes to produce search results. "Guidance is often required, especially for novice users, on what visualizations (scatterplot, parallel coordinates, treemaps, etc.) are appropriate for a given task; the focus should be on the problem to be solved rather than the type of raw data." (Keim et al. 2010, p. 123). To facilitate use and understanding, domain-specific presentation methods need to be provided along with strategies for improving the ability to describe and communicate ideas (Roth-Berghofer and Richter 2008). In addition, approaches are needed that present the user not just with results, but also show the often numerous and interdependent criteria in a transparent and understandable way (Gratzl et al. 2013) so that the user can adjust these as required. Keim summarizes these requirements as follows:

- "Progressive analysis: provide quick answers first, then make improvements incrementally or on-demand;

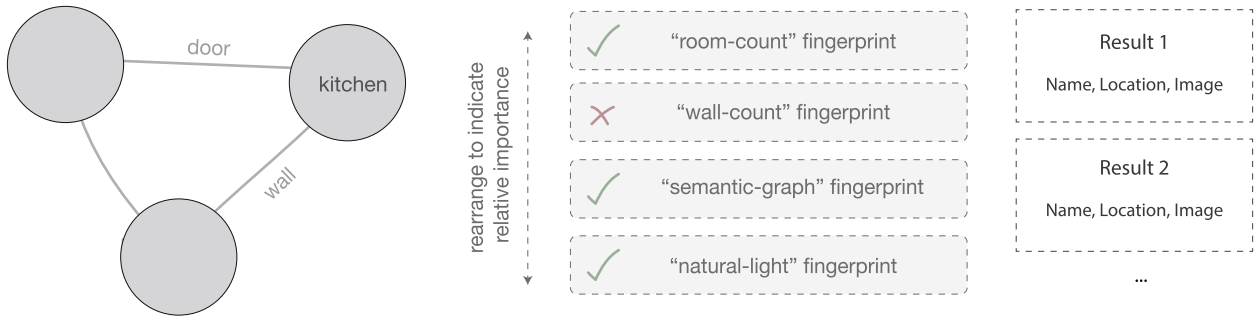


图2 左侧为从草图中提取的简化图，右侧为所选的指纹特征。

该组件负责处理查询并将查询的各部分分发至各个微服务。第六，协调器可为客户端应用程序预先计算可视化数据。

软件架构

为实现语义搜索引擎，我们提出了一种基于联邦信息系统的数据集成方案。在该系统中，不同的自治信息源（如BIM服务器、图数据库和内容管理系统）可通过统一的XML模式进行集成，并以REST方式作为单一信息源进行查询。这使得能够根据各类信息的特定属性对其进行高效存储与处理。与传统的数据仓库方法不同，联邦信息系统并不对各数据源进行复制，而是通过处理组件分别对各源进行查询，并将查询结果汇集起来，供客户端应用中的协同组件系统进一步处理。对于离线处理及建筑平面图指纹的提取，需要两类组件：a）用于分析并扩充非结构化数据源（CMS）的组件；b）用于扩展并为结构化数据源（BIM）中的概念赋予属性的组件。图3展示了该信息系统的软件架构，其借鉴了2012年文献（第461页）中语义搜索引擎的软件架构。拓扑信息通过多种方法从正式（BIM）与非正式（CMS）数据源中离线提取，并以图谱（指纹）的形式加以形式化。离线处理必须对数据进行合理组织，以便在在线处理过程中，用户能够在可接受的时间范围内获得有用的结果。

所示组件为信息云（服务器）中以及不同终端设备（客户端）上的若干独立应用程序的集合。在服务器端，该系统由数据存储、处理与协调组件构成。可能的客户端包括配备鼠标和键盘的台式计算机（如办公室中的固定工作站），以及平板电脑（如便携式设备，例如用于客户拜访和会议）、智能手机（移动设备，例如现场使用或现场调研）以及多点触控桌（固定式设计环境）。

各类终端设备上的应用程序会向信息云平台进行查询（图4）。用于在设计早期阶段辅助设计师的信息系统由一系列数据存储与处理组件构成，旨在存储并处理语义化的建筑模型数据、图数据以及文本数据。

应用情境与个体设计过程因具体案例而异。因此，有必要采用不同的抽象层次，以使设计师能够尽可能完整、准确地表达其心智模型。由于在设计的早期阶段，设计师往往对设计方案仅有模糊且不完整的认知，系统需具备高度的表达自由度及多样化的抽象方式。

不同的抽象层次

系统必须提供交互方式，使用户能够表达其心智模型，并将其与存储的模型模式进行分析和比较，从而生成搜索结果。“尤其是在面向新手用户时，往往需要指导其针对特定任务应选用何种可视化方法（如散点图、平行坐标图、树状图等）；关注点应放在待解决的问题本身，而非原始数据的类型上。”（Keim 等，2010，第123页）。为便于使用与理解，还需提供领域特定的呈现方法，并辅以提升用户描述与交流思想能力的策略（Roth-Berghofer 和 Richter，2008）。此外，还应采用能够不仅向用户提供结果，而且以透明且易于理解的方式展示众多且相互关联的评价标准的方法（Gratzl 等，2013），以便用户根据需要对这些标准进行调整。Keim 将上述需求概括如下：

- 渐进式分析：先提供快速解答，随后按需或逐步进行改进；

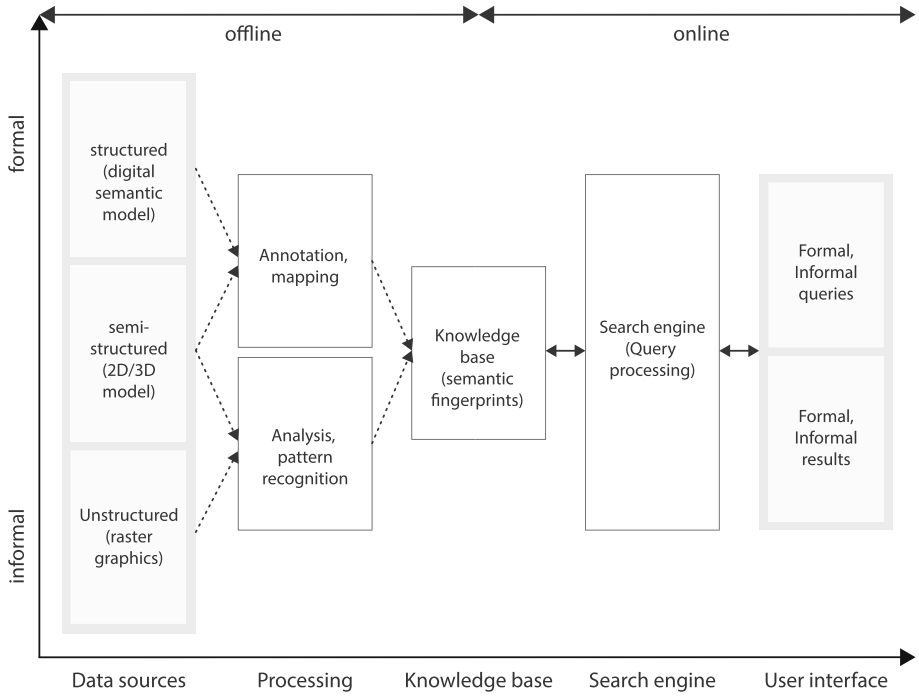


Fig. 3 Software architecture of semantic information systems based loosely on Dengel (2012)

- Management of dynamic data: incremental analysis instead of restarting it from the beginning;
- Steerable analysis: allow long-computations to be steered by users when possible." (Keim et al. 2010, p. 106)

The aim is to gradually narrow down possible solutions or reference cases by providing ongoing feedback so that the query can be successively steered and defined by the user. An example of this is shown in Fig. 5 for building information in the early design phases. The degrees of detail shown are the product of thinking processes that gradually transform a design and make it more concrete.

Corresponding interaction strategies for geometric, topological, geographic and lexical information are likewise required. In the information system we have implemented, we differentiate between five main representations for formalizing mental models according to their respective characteristics:

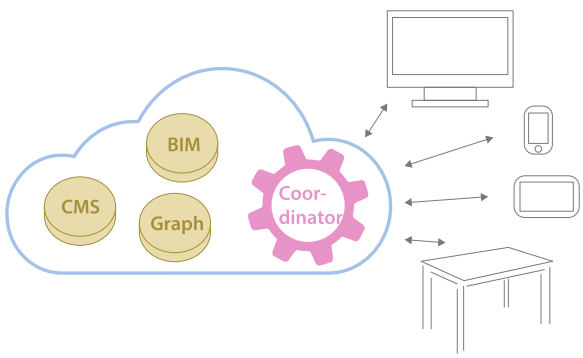


Fig. 4 Information cloud

- Texts (for example architect, year of completion, building costs, text search, descriptions, building typology)
- Tables (for example schedule of rooms, schedule of works, cost plan, specification of works, list of neighbors, access routes)
- Schemes (for example diagrams, spatial arrangement, zones, orientation, proportions, passage through a room, adjacent relation to other spaces)
- Freehand sketches (for example, arrangement of spaces, zoning, orientation, proportions, passage through a room, spatial delineation, floor plans, elevations, sections)
- 2D/3D drawings (for example, arrangement of spaces, zoning, orientation, proportions, passage through a room, adjacency relationships, cubature, floor plans, elevations, sections, perspectives).

The transformation between different digital formalization's and visualizations and their respective levels of abstraction is of particular help in the design process and is a topic of ongoing research. The aim is to examine the technical possibilities for computer-based interpretation of user input and the visualizations of building data.

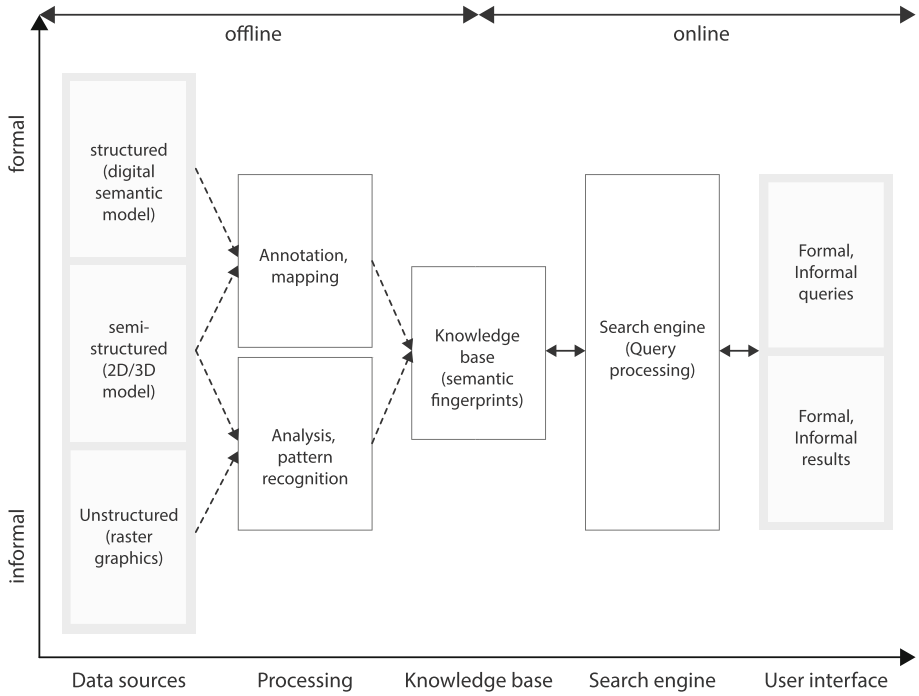


图3 软　基于邓格尔的语义信息系统的软件架构　012)

- 动态数据的管理：采用增量分析，而非从头重新启动；
- 可引导分析：在可能的情况下，允许用户对耗时较长的计算过程进行干预与引导。（Keim 等，2010，第106页）

其目的在于通过持续提供反馈，逐步缩小可能的解决方案或参考案例的范围，从而使用户能够对查询进行逐次引导与界定。图5展示了这一过程在建筑设计早期阶段的应用示例；所呈现的细节层次是设计思维逐步演进、使方案不断具体化的结果。

针对几何、拓扑、地理及词汇信息的相应交互策略是同样也是必需的。在我们所实现的信息系统中，我们根据其各自的特点，将用于形式化心智模型的主要表示方法区分为五类：

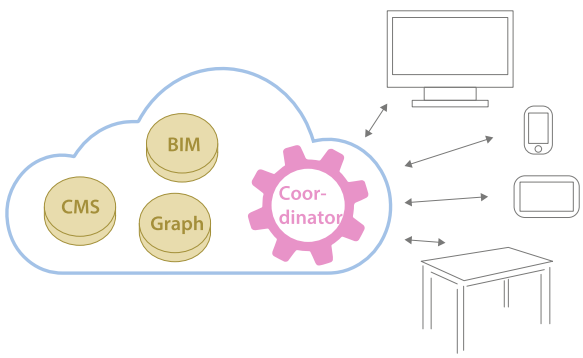


图4 信息云

- 文本（例如建筑师、竣工年份、建筑成本、文本检索、描述、建筑类型）
- 表格（例如：房间安排表、工程进度计划表、成本预算表、工程规范说明、邻近单位清单、交通路线图）
- 方案（例如，示意图、空间布局、功能分区、朝向、比例关系、室内流线、与其他空间的毗邻关系）
- 手绘草图（例如，空间布局、功能分区、朝向、比例关系、室内流线、空间界定、平面图、立面图、剖面图）
- 二维/三维图纸（例如，空间布局、功能分区、朝向、比例关系、室内流线、邻接关系、体量关系、平面图、立面图、剖面图、透视图）。

不同数字化形式化与可视化及其各自抽象层次之间的转换，在设计过程中具有重要辅助作用，并且仍是当前研究的热点课题。研究旨在探讨基于计算机对用户输入进行解析以及对建筑数据进行可视化的技术可能性。

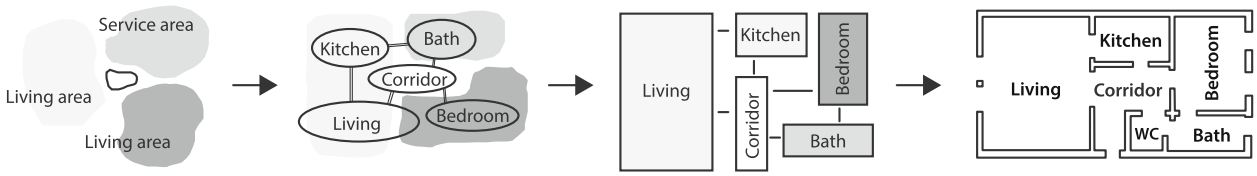


Fig. 5 Typical presentation schemes and their possible transformations, based loosely on (Hanson 1998, p. 308) and (Liebich 1994, p. 75)

Prototypes

Our prototypes enable users to interactively retrieve descriptions of similar building floorplans by sending queries which are constructed visually to a Coordinator middleware. The ability to add detail to or modify one's input, for example by adding a room to a drawing, makes it possible to formulate more granular fingerprint-queries and the mental model corresponds better to the designer's own mental model. By providing ongoing feedback, e.g. in the form of reference projects that match the fingerprint, the information system prompts the designer to modify and adapt his or her design idea. By increasing or decreasing the degree of specification, the set of possible search results can be expanded or focused to better match the problem at hand. What density of information is adequate and necessary depends on the specific application at hand and the user. For the general definition of spaces, their relationships and the passage between them, a simple graph can be sufficient. Graphs can take the form of diagrams or tables (Beck et al. 2014, p. 88). In our information system, we have used node and edge diagrams to make it easier to depict the location and relationship between rooms or nodes. Our user interface investigations looked at ways of providing vague input about spatial situations and constellations, i.e. their basic characteristics and arrangement. The corresponding formulation of this as a graph is shown to the user at different levels of abstraction.

The system has to provide interaction approaches that allow the user to express the mental model that can be analyzed and compared with stored model schemes to produce search results. Therefore, contextual coherence between the mental model and the representations of information is necessary. Accordingly, suitable strategies need to be implemented to achieve a beneficial outcome for the user.

Whenever the user modifies the sketch or the parameters of the filters, a new query needs to be generated and sent to the Coordinator. The sketch is encoded as an AGraphML floorplan graph and the filters are encoded as XML as well. After the coordinator has received the query, it executes it by distributing parts of it to the connected microservices and combining the results.

The ability to add detail to or modify one's input, for example by adding a room to a drawing, makes it possible to express more granular fingerprint-queries and the mental model corresponds better to the designer's own mental model. For this purpose, we implemented separate prototypes that use the same data stock to compare different input and retrieval strategies.

Our user interface investigations looked at ways of providing vague input about spatial situations and constellations, i.e. their basic characteristics and arrangement. The corresponding mathematical formulation of this as a graph is shown to the user at different levels of abstraction.

The different use cases (e.g. in the office, when visiting a client, in a meeting, out on site or while designing) are conceived as prototypes for the respective user. For the most common office drawing situation with mouse and keyboard, we propose using the *a.vista* concept previously developed by Christoph Langenhan in 2008 (Langenhan and Petzold 2010), which has different semantic ways of describing a building (level, unit, zone, and room).

The *ar.searchbox* (see Fig. 6) is a media reference collection from the *mediaTUM* (Langenhan et al. 2012) content management system that was devised as a research tool for buildings. The university library and computer science students maintain and enhance the system while students of architecture at the TUM enter and maintain data. It is possible to search using conventional textual search queries in the browser and the entries are linked to other information sources, such as the architects' website or a digital building model based on an analysis of the pixel images, building models and links.

For the *metis WebUI* (see Fig. 7), Johannes Bayer at the Deutschen Forschungszentrum für Künstliche Intelligenz (DFKI GmbH) took a platform-independent 2D modelling approach and implemented, among other things, a means of enriching search queries (Bayer et al. 2015). In Fig. 7 one can see the spatial arrangement as a bubble diagram on the left and actual spatial arrangement with symbols for doors and windows on the right, demonstrating the different formalizations of levels of abstraction and detail.

An approach for formalizing queries as freehand sketches was developed by Markus Weber in 2009 (Weber et al. 2010) with *a.scatch* (Fig. 8). Doors are drawn as double lines and adjacent walls as single lines. The rooms (Fig. 8, left a) are recognized and the room type labelled by hand (Fig. 8, left c). The resulting room (Fig. 8, left b) and

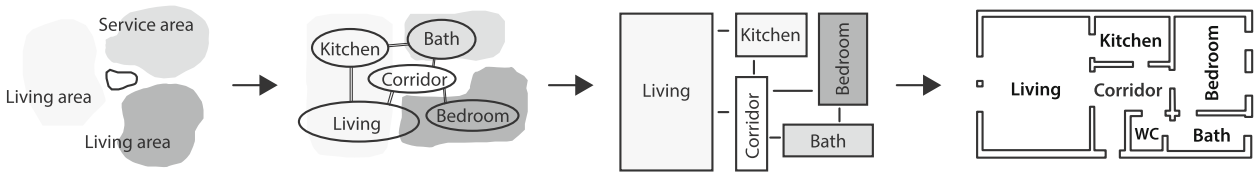


图5 典型表现 方案及其可能的变换，大致基于（Hanson 1998，第…页）。 308）和（Liebich 1994，第75页）

原型

我们的原型系统使用户能够通过向协调中间件发送以可视化方式构建的查询，交互式地检索相似建筑平面图的描述。用户可以对输入逐步细化或进行修改，例如在图纸中添加房间，从而形成更为精细的指纹查询；这一设计也使其认知模型与设计师自身的思维模式更加契合。通过持续提供反馈，例如以与指纹匹配的参考项目形式呈现，信息系统能够引导设计师不断调整和完善其设计方案。通过增减信息的细化程度，可相应扩展或聚焦可能的搜索结果集，使其更贴合当前的具体问题。所需信息的密度应视具体应用场景及用户需求而定；对于空间的一般性定义、空间之间的相互关系及其连通性，简单的图结构即可满足要求。这些图可以采用示意图或表格的形式（Beck 等，2014，第88页）。在我们的信息系统中，我们使用节点—边图来更直观地表达房间或节点之间的位置关系。我们的用户界面研究探讨了如何以模糊的方式输入空间情境及其构型，即其基本特征与布局；相应的图结构表述会以不同抽象层次呈现在用户面前。

系统必须提供多种交互方式，使用户能够表达其心理模型，并将其与存储的模型模式进行分析和比对，从而生成检索结果。因此，心理模型与信息表征之间必须保持语境一致性。相应地，还需实施恰当的策略，以确保为用户带来有益的使用体验。

每当用户修改草图或滤波器的参数时，都需要生成一个新的查询并将其发送至协调器。其中，草图被编码为AGraphML格式的版图图，滤波器同样以XML格式进行编码。协调器在接收到该查询后，会通过将查询的不同部分分发给所连接的微服务，并对各服务返回的结果进行整合来执行该查询。

能够对用户的输入进行细节补充或修改，例如在图纸中添加一个房间，这使得……成为可能。

以表达更为细粒度的指纹查询，并使心智模型更好地与设计者自身的心智模型相契合。为此，我们实现了若干独立的原型，这些原型采用相同的数据集来比较不同的输入与检索策略。

我们的用户界面研究探讨了如何以模糊的方式表达空间情境及其构型，即其基本特征与排列方式。与此相关的图论形式化表示会在不同抽象层次上呈现给用户。

不同的使用场景（例如在办公室、拜访客户时、会议中、现场或进行设计时）被构想为针对相应用户的原型。对于最常见的配备鼠标和键盘的办公绘图情境，我们建议采用克里斯托夫·朗根汉于2008年提出的a.vista概念（Langenhan和Petzold，2010），该概念以楼层、单元、区域和房间等不同语义维度来描述建筑。

ar.searchbox（见图6）是mediaTUM内容管理系统（Langenhan等，2012）中的一个媒体资源集合，该系统被设计为面向建筑领域的研究工具。慕尼黑工业大学的大学图书馆与计算机科学专业的学生负责维护并持续完善该系统，而该校建筑学专业的学生则负责数据的录入与更新。用户可通过浏览器使用传统的文本检索查询进行搜索，且各条目均与其他信息源相链接，例如建筑师的官方网站，或基于像素图像、建筑模型及关联关系构建的数字化建筑模型。

对于 metis WebUI（见图7），德国人工智能研究中心（DFKI GmbH）的约翰内斯·拜耳采用了一种与平台无关的二维建模方法，并实现了包括丰富搜索查询在内的多项功能（Bayer et al., 2015）。在图7中，左侧展示了以气泡图形式呈现的空间布局，右侧则为包含门和窗符号的实际空间布局，从而体现了不同抽象层次与细节程度的形式化表达。

Markus Weber 于2009年提出了一种将查询形式化为手绘草图的方法（Weber等，2010），并开发了a.scatch工具（图8）。门被绘制为双线，相邻墙体则以单线表示。房间（图8，左a）被识别，并由人工标注房间类型（图8，左c）。由此生成的房间（图8，左b）以及

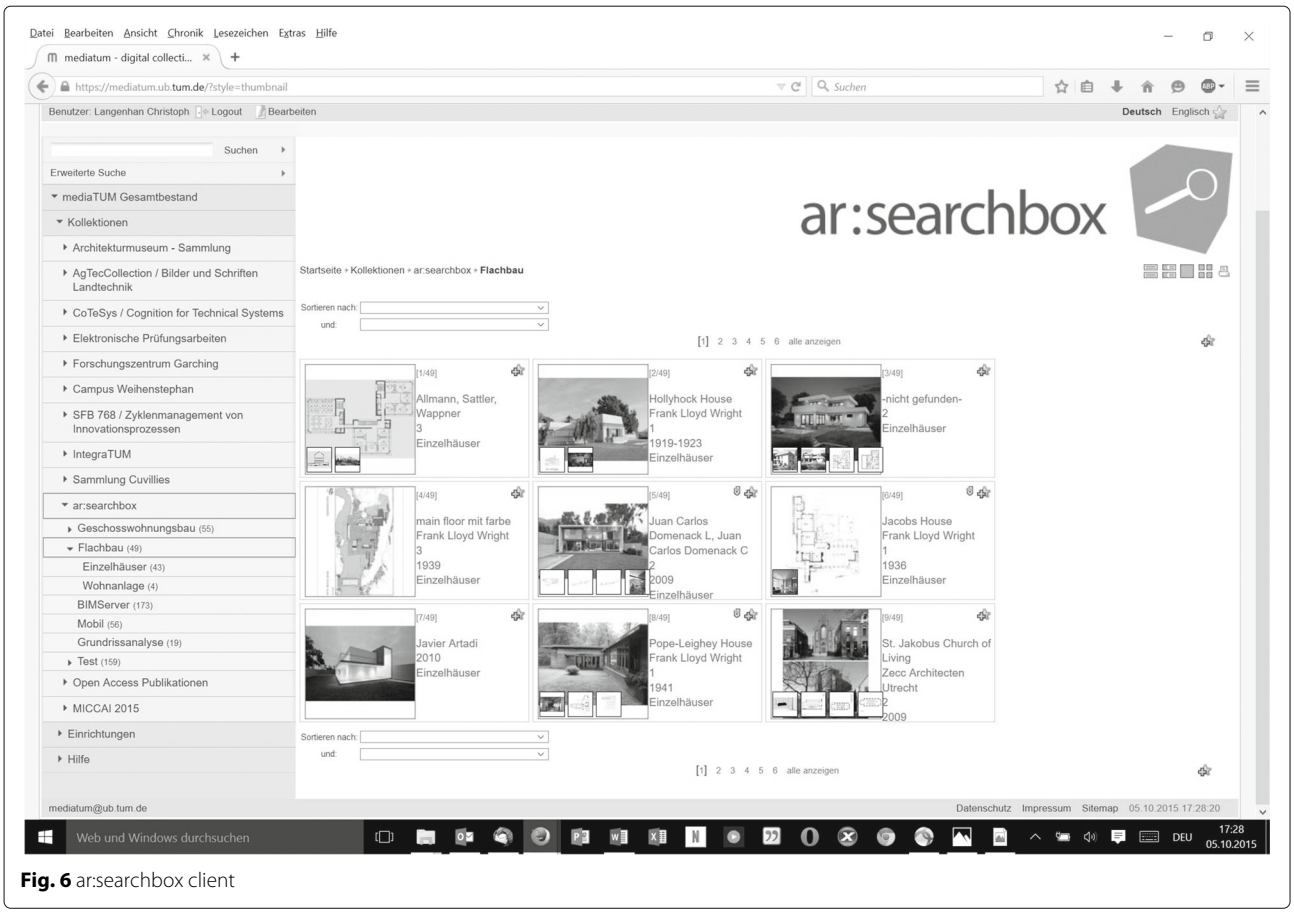


Fig. 6 ar:searchbox client

be used for the search query and a list of corresponding search results is then displayed (Fig. 8, right).

The application *Touchtect* Fig. 9 by Thomas Kinnen and Dario Banfi for a multi-touch table combines freehand drawing, geographic input and meta information from the ar:searchbox (Fig. 6). Here, however, the room type is selected from a predefined list, while the application *ar:searchDroid* (Fig. 10) by Sebastian Seitz is designed for a tablet computer for mobile research and can restrict search results to buildings that are in the vicinity.

All of the above applications for freehand drawings do not detect what has been drawn by visually processing the results but by detecting the movement of the stylus. As such, a polygon denoting a room needs to be closed in order to be recognised for use in the spatial query. To afford the user greater flexibility while drawing, the *cormorant* application (Fig. 11) by Dominic Henze uses a program library (Vatavu et al. 2015) for gesture recognition developed by the University of Washington. The application *ar:searchPad* by Jana Pejic (Fig. 12) does away with the input of geometric figures altogether. This application is based entirely around the input and comparison of bubble diagrams.

To incorporate such topological approaches in the design process, Thomas Stocker, Dario Banfi, Jana Pejic, Thomas Kühner, Markus Dausch, Bishwa Hang Rai, Dominic Henze, Arno Schneider and Johannes Roith have jointly developed an Add-on called *Dolphin* (Langenhan and Petzold 2015) for *Rhino3D* and its parametric design extension *Grasshopper3D*. The *Dolphin* Add-on provides a series of components for visual programming using *Grasshopper3D* that make it possible to query the information system using drawings in *Rhino3D* and additional meta information. The data can be exported for further use directly from the graph database using a service called *pigeon* developed by Leon Höß and Christopher Will. For example, AgraphML files can be imported with other components, either individually or as a collection, into *Grasshopper3D* for further use.

The Coordinator to prepare and visualize the results

The middleware described below is implemented as a RESTful web service. Clients pass queries to the Coordinator as XML via HTTP. The system executes a query using one or more agents. While the query is being executed, a result list is maintained on the server and continuously

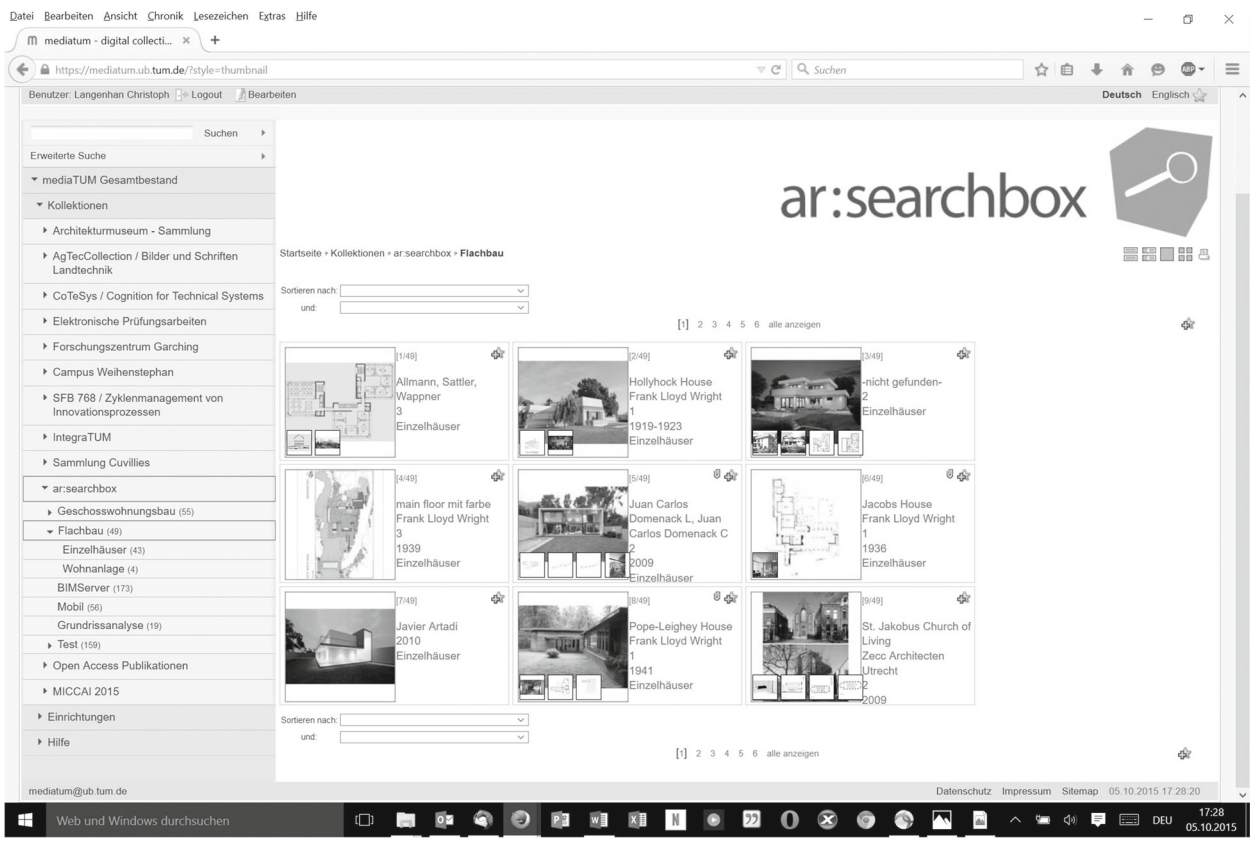


图6 ar:搜索框客户端

用于搜索查询，随后显示相应的搜索结果列表（图8，右侧）。

Thomas Kinnen 和 Dario Banfi 开发的多点触控桌应用 Touchtect（图9）将手绘、地理信息输入与来自 ar:searchbox（图6）的元数据相结合。然而，在该应用中，房间类型需从预定义列表中选择；而 Sebastian Seitz 开发的 ar:searchDroid 应用程序（图10）则专为平板电脑设计，用于移动式检索，并可将搜索结果限定在周边建筑范围内。

上述所有用于徒手绘图的应用程序均不是通过视觉处理绘制结果来识别所绘内容，而是通过检测手写笔的运动轨迹来实现。因此，用于空间查询的房间多边形必须闭合才能被正确识别。为使用户在绘图时拥有更大的灵活性，Dominic Henze 开发的 Cor-morant 应用程序（图11）采用了由华盛顿大学开发的、用于手势识别的程序库（Vatavu 等，2015）。而 Jana Pejic 开发的 ar:search Pad 应用程序（图 12）则完全摒弃了几何图形的输入方式，该应用程序完全基于气泡图的输入与比对。

为了在设计过程中引入此类拓扑方法，Thomas Stocker、Dario Banfi、Jana Pejic、Thomas Kühner、Markus Dausch、Bishwa Hang Rai、Dominic Henze、Arno Schneider 以及 Johannes Roith 共同为 Rhino3D 及其参数化设计扩展 Grasshopper3D 开发了一款名为 Dolphin 的插件（Langenhan 和 Petzold，2015）。该 Dolphin 插件提供了一系列基于 Grasshopper3D 的可视化编程组件，使用户能够通过 Rhino3D 中的图纸及附加元信息对信息系统进行查询。数据可通过 Leon Höß 和 Christopher Will 开发的名为 pigeon 的服务直接从图数据库导出，以供进一步使用。例如，其他组件可将 AgraphML 文件作为单个文件或集合导入 Grasshopper3D，以便后续处理。

协调员负责结果的准备与可视化

下文所述的中间件被实现为一个RESTful Web服务。客户端通过HTTP以XML格式向协调器传递查询请求。系统利用一个或多个代理来执行查询。在查询执行过程中，服务器端会维护一个结果列表，并持续地

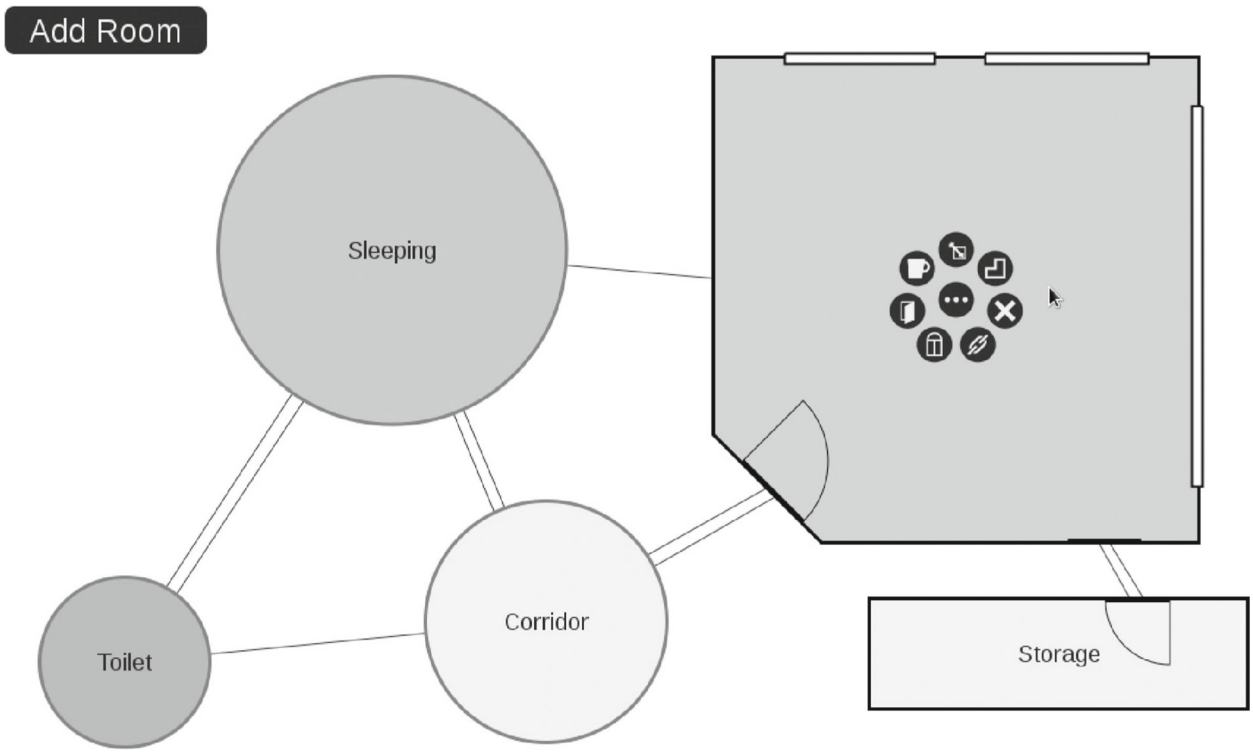


Fig. 7 Metis Web UI client

updated. The client receives a stream of change events until the execution terminates. Possible events include insert, remove and update events. The basic data flow is shown in Fig. 13.

Query processing

The server provides various types of conditions, such as a metadata conditions which can be used to filter results based on simple attributes such as the name of the architect. A very simple example would be the room-count condition which finds all building floorplans in the database that have the same number of rooms as the floorplan graph in the query. We have currently defined 12 fingerprints that check, for example, properties of nodes, edges or full graph isomorphism (Weber et al. 2011). Figure 14 shows some example queries. Below we shortly describe the query process. Detailed information can be found in Roith et al. (2016).

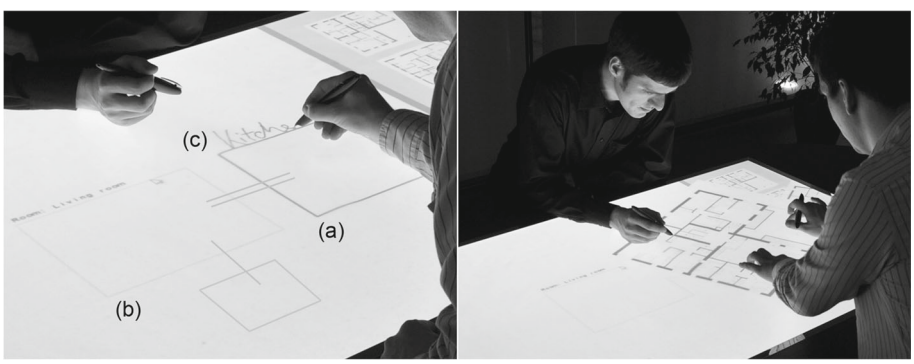


Fig. 8 a.scatch: Sketch-Based Retrieval for Architectural Floor Plans

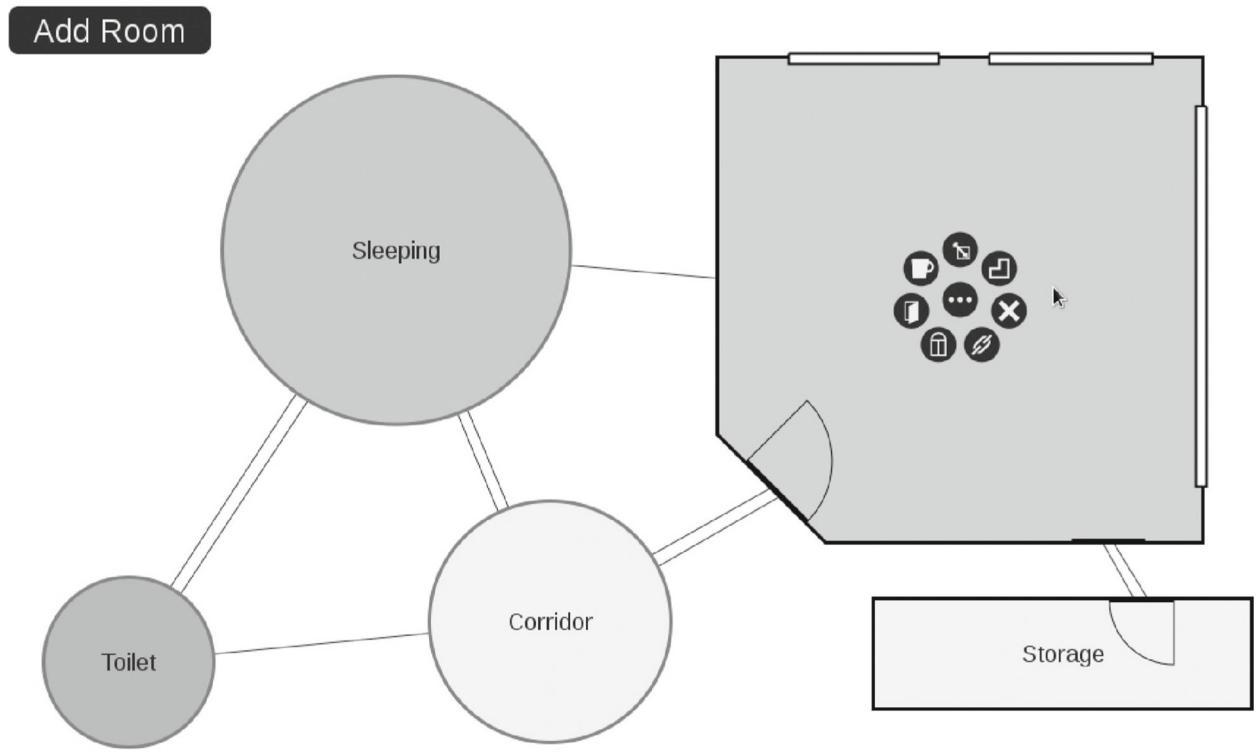


图7 Metis Web UI 客户端

已更新。客户端会持续接收变更事件流，直至执行终止。可能的事件类型包括插入、删除和更新事件。基本数据流如图13所示。

查询处理

该服务器提供多种类型的条件，例如元数据条件，可用于根据诸如名称之类的简单属性对结果进行筛选。架构师。一个非常简单的示例是房间数条件，该条件可在数据库中查找与查询中的楼层平面图具有相同房间数的所有建筑楼层平面图。目前我们已定义了12种指纹特征，用于检测诸如节点属性、边属性或全图同构等特征（Weber 等，2011）。图14展示了一些示例查询。下文将简要描述查询过程，详细信息请参见 Roith 等（2016）。

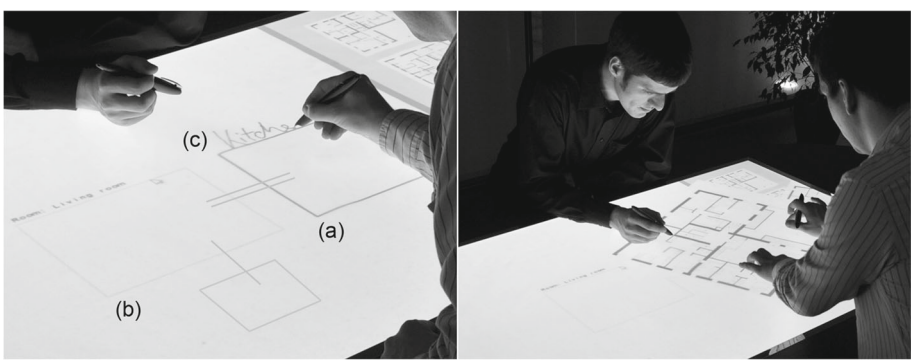


图8 a.scatch：基于草图的建筑平面图检索



Fig. 9 Touch tech client

Queries are executed in multiple phases. The basic process is shown in Fig. 15. The query arrives as XML at the service layer of the application and is parsed into a tree (AST, Abstract syntax tree). Since our queries are simple, we do not generate a query plan but simplify and normalize the tree using a set of rewriters: This also simplifies caching by making it easier to detect semantically equal[2] queries. The best agent is picked using a set of rules from a rule base, which is currently maintained manually by an expert. The simplified AST is then used to create *data sources* which are instances of agents (Fig. 16). Furthermore, a cached data source from a previous execution can often be reused. Next, all data sources are executed in parallel. Agents produce a stream of objects which represent building floorplans or storeys and can be uniquely identified. Time limits imposed by the configuration are used to abort agents that exceed their allocated execution time. Similarly, agents self-terminate or are aborted when they produce more than a certain number of results. While passing through the DAG,

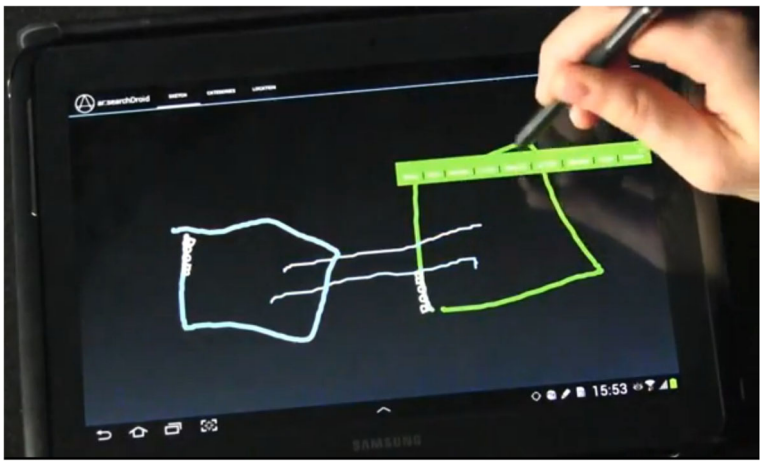


Fig. 10 User interface ar:searchDroid



图9 触控技术客户端

查询在多个阶段中执行。基本流程如图15所示。查询以XML格式到达应用程序的服务层，并被解析为一棵抽象语法树（AST）。由于我们的查询较为简单，我们不生成查询计划，而是通过一组重写规则对这棵树进行简化和规范化；这一过程也有助于缓存的管理，因为使得语义等价的查询更容易被识别。最佳代理则根据规则库中的一组规则进行选择，该规则库目前由专家手动维护。简化的抽象语法树随后用于创建数据源，这些数据源是代理的实例（图16）。此外，先前执行过程中生成的缓存数据源通常可以被复用。接下来，所有数据源并行执行。代理会生成一系列对象，这些对象代表建筑的平面图或楼层，并且具有唯一标识。配置中设定的时间限制用于终止那些超出其分配执行时间的代理。同样地，当代理产生的结果数量超过某一阈值时，它们也会自行终止或被强制中止。在沿着有向无环图（DAG）传递的过程中，

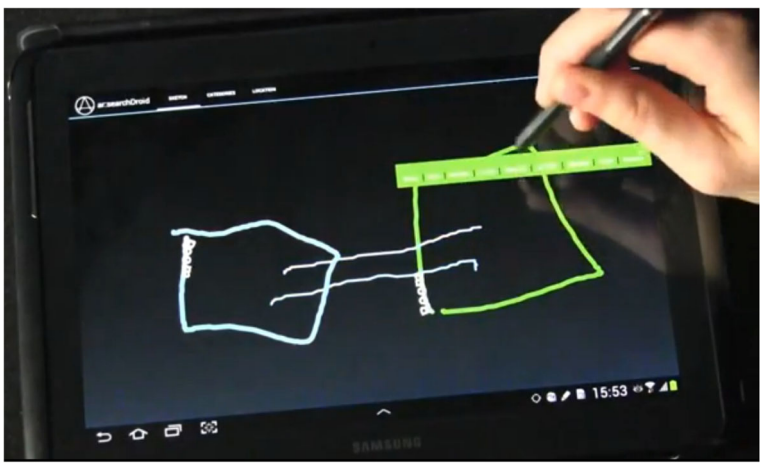


图10 ar:searchDroid用户界面

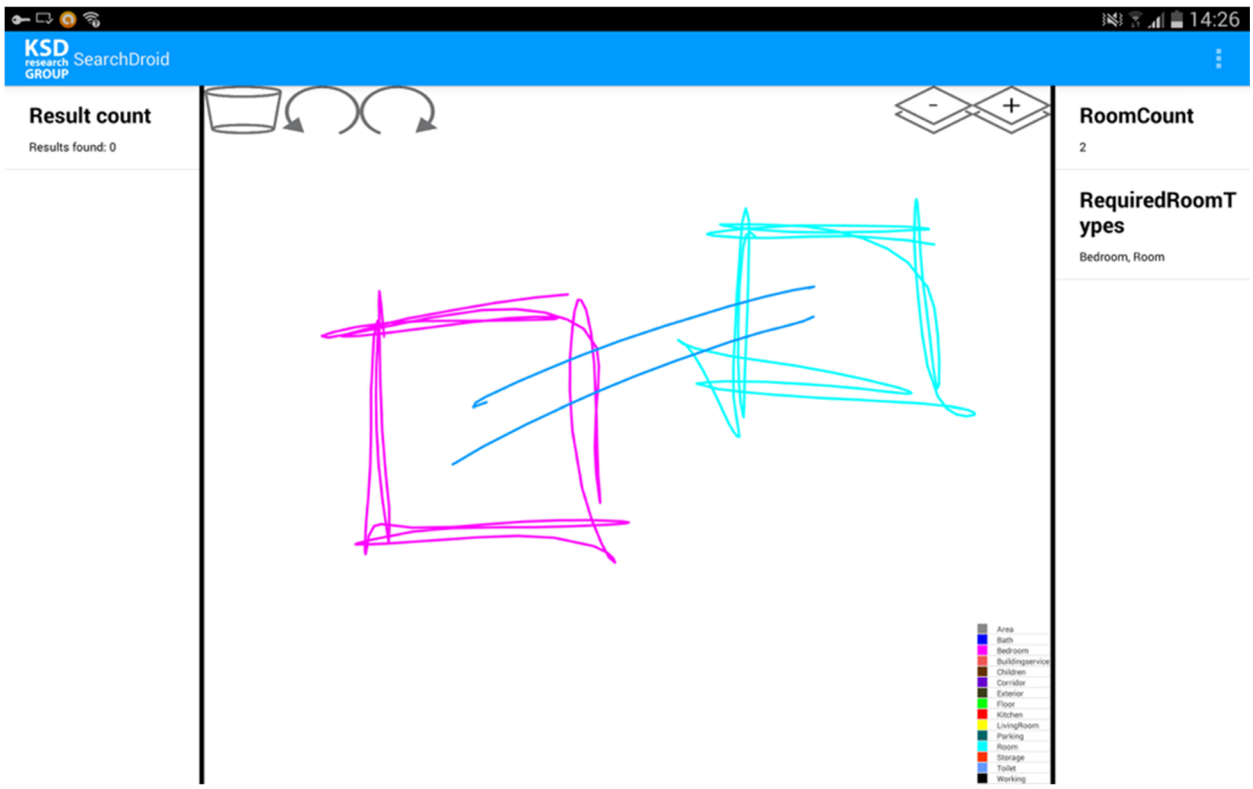


Fig. 11 User interface of 'cormorant'

results can be merged or dropped. Results that fulfill all conditions are inserted into a results list that is sorted by the similarity score of the results. Clients observing the list can specify whether they want to receive dynamic updates or a static list. Any changes made to the list, or to the content of items, are then propagated to the clients using appropriate change events. In the case of a static list, no events are sent until the execution has finished, resulting in an XML stream of only 'append' elements that can be interpreted as a document. Clients can specify which data they want to receive for the results. For example, each building/storey usually has associated metadata, a floorplan graph or some images. This data is not provided by agents. Instead the Coordinator has a concept of data model providers which fetch and provide any data content requested and deliver preprocessed data for visualization.

Although clients can specify the list of possible or required fingerprint matches as part of the expression tree, it is also possible to allow the server to select appropriate fingerprints based on the query graph. In this case, clients include a special 'auto' match condition in the query tree. For example, if a floorplan graph lacks node or edge labels, a fingerprint that relies on those labels can't be computed and will not be included in the server-selected list. Fingerprints may also have parameters, for example to adjust the required precision of a match. For server-selected fingerprints, appropriate parameters need to be selected. In some cases, it may be appropriate to select multiple possible parameters and have multiple instances of the same fingerprint. For example, the room-area fingerprint is always parameterized by the room type. If created during server-side fingerprint selection, a fingerprint instance is added for each room type that is found in the query graph.

For each condition (and, by extension, fingerprint) an agent must be selected to execute it. If more than one agent is compatible with the condition/fingerprint type and parameters, a good one (that produces good results, ranks them well, executes quickly and consumes little resources) has to be picked. The rules usually look at certain features of the floorplan graph - for example the number of nodes - to make a decision.

If an AND or OR node contains multiple children that are assigned to different agents, the node itself does not form an assignable subtree but it is often possible to extract several children that share the same agent into a new node of the same type and make the newly created

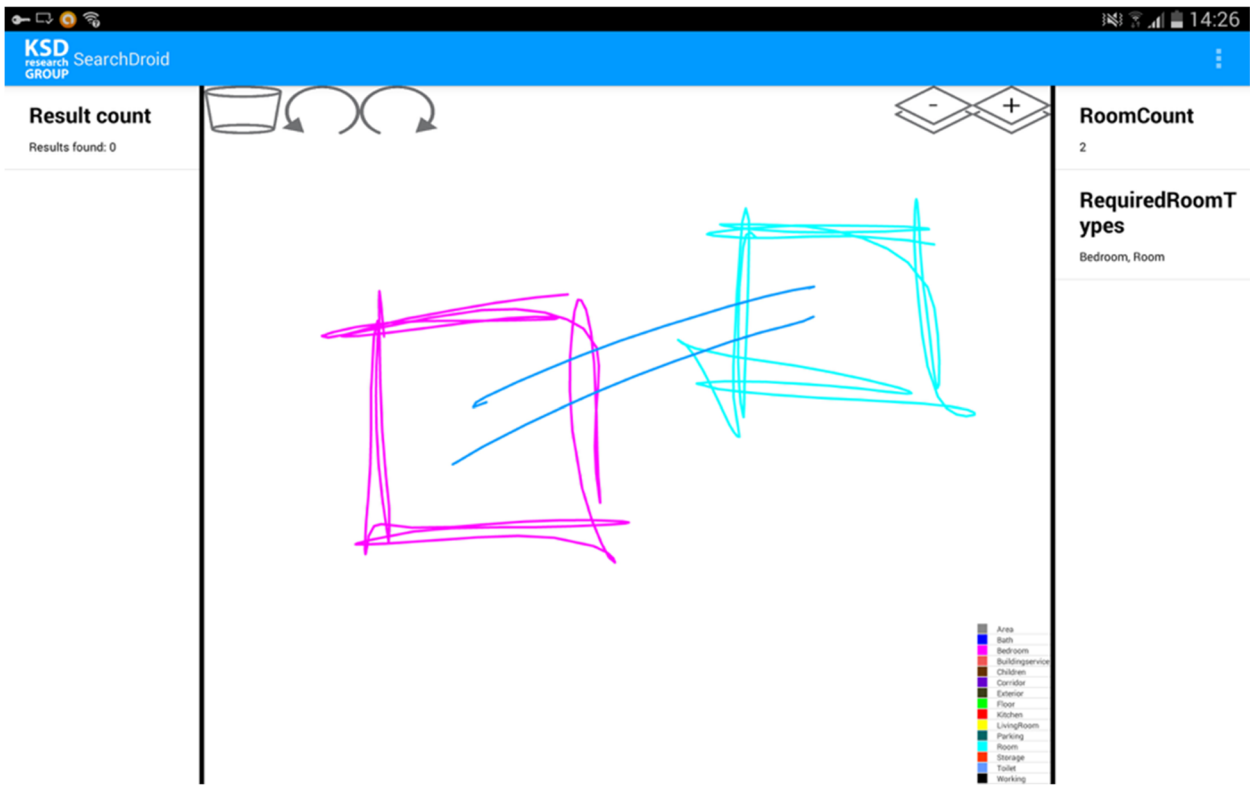


图11 “cormorant”用户界面

结果可以被合并或丢弃。满足所有条件的结果将被插入到一个按相似度得分排序的结果列表中。观察该列表的客户端可以指定是接收动态更新还是静态列表。对列表或其中条目内容所做的任何更改，都会通过相应的变更事件传播给客户端。对于静态列表，在执行结束之前不会发送任何事件， resulting in an XML stream of only ’append’ elements that can be interpreted as a document. 客户端可以指定希望接收哪些结果数据。例如，每个建筑或楼层通常都关联有元数据、楼层平面图或若干图像。这些数据不由代理提供，而是由协调器管理的数据模型提供者负责获取并提供所请求的数据内容，并为可视化处理预处理后的数据。

尽管客户端可以在表达式树中指定可能的或必需的指纹匹配列表，但也可以允许服务器根据查询图选择合适的指纹。在这种情况下，客户端会在查询树中包含一个特殊的“自动”匹配条件。例如，如果楼层平面图缺乏节点或边的标签，则依赖这些标签的指纹无法计算的指纹将不会被纳入服务器选定的列表中。指纹也可能包含参数，例如用于调整匹配所需的精度。对于由服务器选定的指纹，需要选择合适的参数。在某些情况下，可能需要同时选择多个可行的参数，并为同一指纹生成多个实例。例如，房间面积指纹始终以房间类型作为参数；如果在服务器端进行指纹选择时创建，则会为查询图中出现的每一种房间类型各添加一个指纹实例。

对于每一种条件（进而对应于每一种指纹），都必须选择一个代理来执行该条件。如果存在多个与该条件或指纹的类型及参数兼容的代理，则需要从中挑选出一个性能优良的代理——即能够产生良好结果、对结果进行合理排序、执行速度快且资源消耗低的代理。规则通常会考察版图图的一些特定特征，例如节点数量，以作出决策。

如果一个与节点或或节点包含多个被分配至不同代理的子节点，则该节点本身并不构成一个可分配的子树，但通常可以将其中属于同一代理的若干子节点提取出来，形成一个同类型的新的节点，并使新创建的

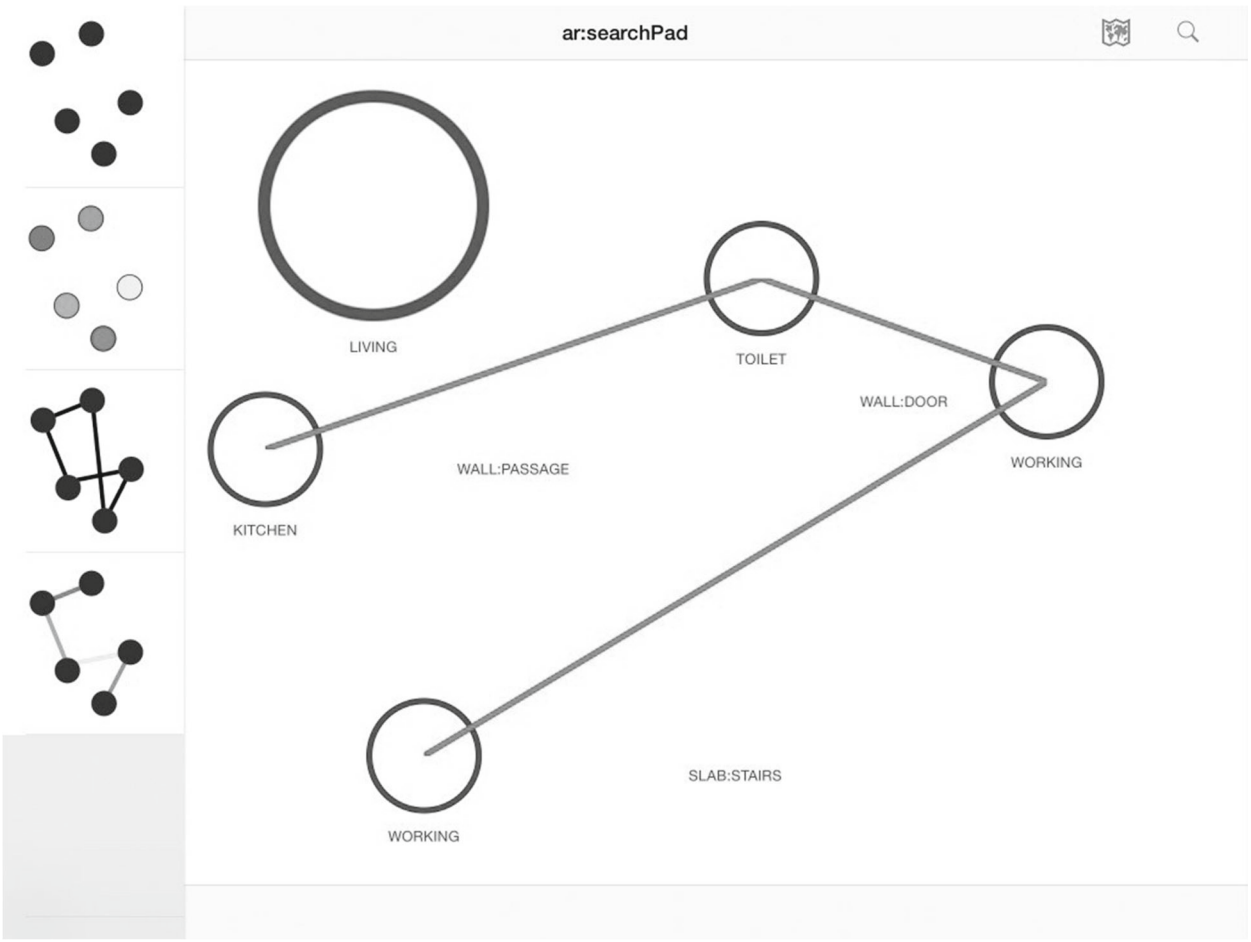


Fig. 12 Another client using bubble diagrams

node a child of the original AND/OR node. Some agents may support subtrees, but not for all types of conditions/fingerprints they support or not if those conditions occur in the same tree. When subtrees are formed via node extraction, agents receive the set of all children of the containing AND/OR node that have been assigned to that agent. They can then cherry-pick the nodes that can be executed together, returning zero or more compatible subsets which can be extracted as assignable subtrees (Fig. 17).

Query execution

After query preparation, all server- and client-side decisions that can affect the returned result set have been taken (based on rules and the server configuration) and specified as part of the rewritten query. The next phase deals with executing queries efficiently.

When combining intermediate result sets, duplicates need to be merged. All elements need to have a common identifier that is available in all data sources. These identifiers are encapsulated in packets, which can carry

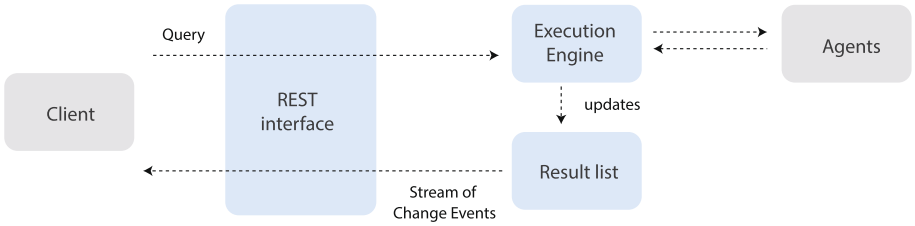


Fig. 13 Overview of system components

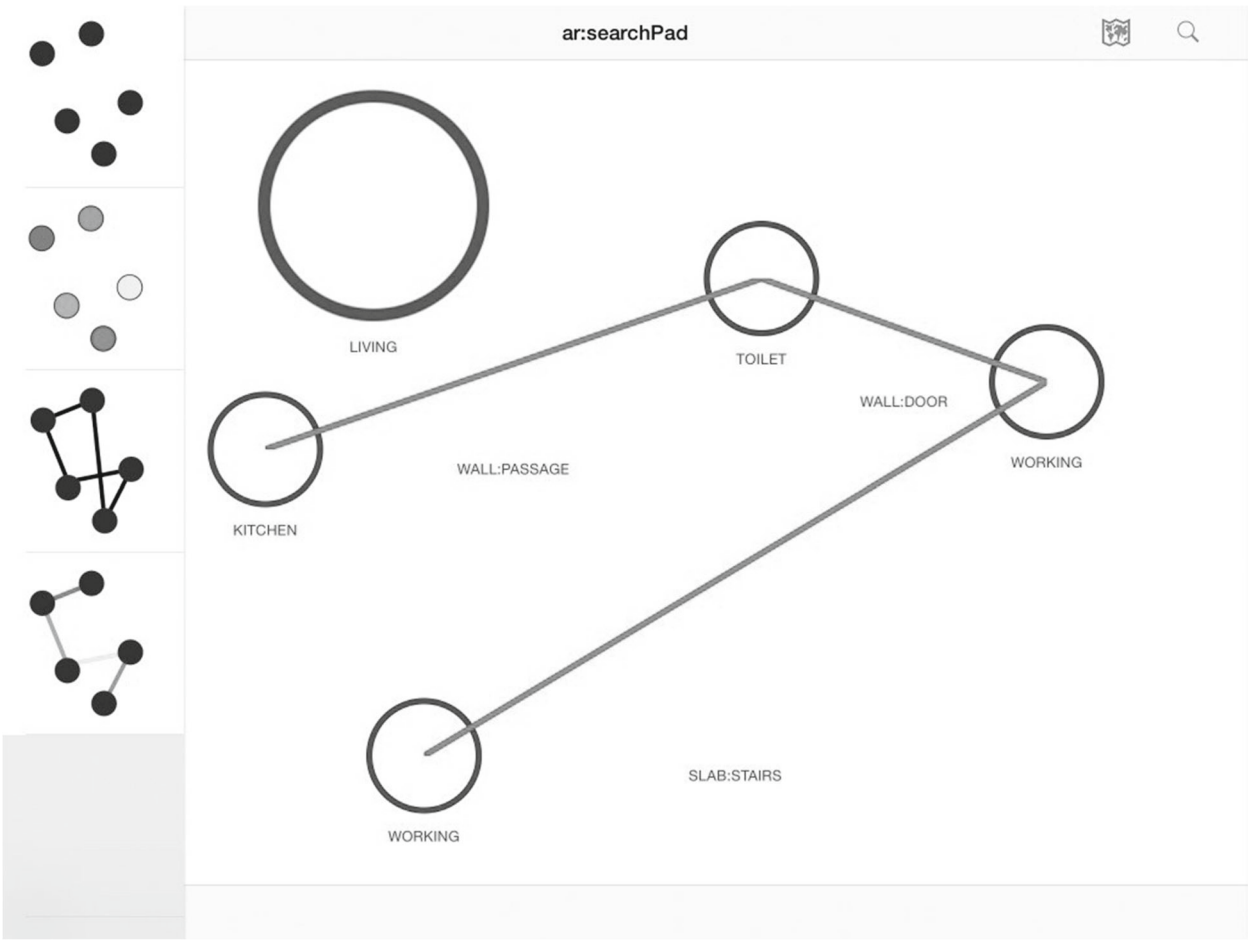


图12 另一位客户使用气泡图

该节点是原始与/或节点的一个子节点。某些代理可能支持子树，但并非对其所支持的所有类型条件或指纹均适用，且当这些条件出现在同一棵树中时也可能不适用。当通过节点提取形成子树时，代理会接收到包含该节点的与/或节点的所有已分配给该代理的子节点集合。随后，代理可从中挑选出能够协同执行的节点，并返回零个或多个可作为可分配子树提取的相容子集（图17）。

查询执行

在查询准备阶段之后，所有可能影响返回结果集的服务器端和客户端决策（基于规则及服务器配置）均已作出，并作为重写后查询的一部分予以明确。下一阶段则致力于高效地执行查询。

在合并中间结果集时，需要对重复项进行去重。所有元素必须具有一个在所有数据源中均可用的共同标识符。这些标识符被封装在数据包中，该数据包可以携带

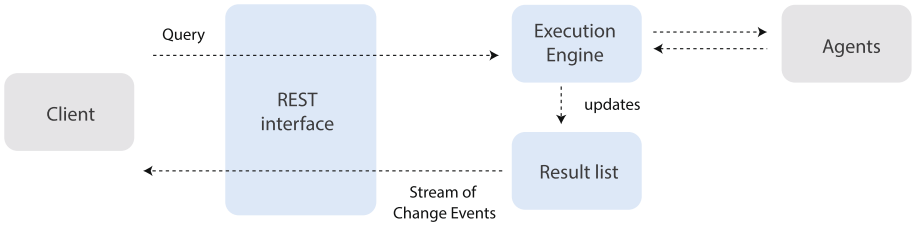


图13 系统组件概览

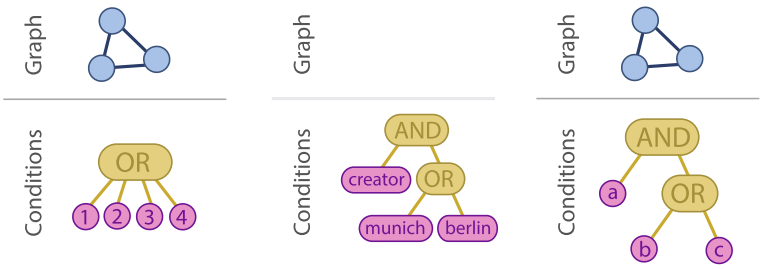


Fig. 14 Example queries: a query with conditions, a graph with fingerprints and a more complex combination of both

associated information, including the set of fingerprints matched so far and the current score of a result. Merges occur while results move through the execution DAG or when a result is added to the result list and an identical result is already present. For example, if two results with the same identity arrive at an AND node, they are combined before being pushed onwards. Packets support a content-specific merge operation. For example, the associated score values are merged into a new combined score.

All conditions in a query have two optional attributes 'fixed' and 'weight' which can be set to an arbitrary rational number by the client which sends the request. If a result is selected by a particular condition, both values are summed and added to the total score for that result. The 'weight' is multiplied by an agent-determined score between 0 and 1 that indicates the quality of the match. Clients can therefore use the 'fixed' value to indicate the importance of the condition independently of the quality, and the 'weight' part to affect the importance of the quality score. Formally, for a result x, the combined rank (before normalization) is currently computed as:

$$rank(x) = \sum_{i=0}^{n} \chi_i(x) \cdot \left(f_i + w_i \cdot s_i(x)\right)$$

where n is the number of conditions, $\chi_i(x)$ is the indicator function for the ith condition, f_i and w_i are the 'fixed' and 'weight' values defined as part of the query and $s_i(x)$ is a score computed by the agent.

Agents as data filters

Agents internally access the complete set of all buildings in the database and produce a subset. Instead of having agents operate on the underlying database set, it would be useful to get them to operate as filters on intermediate

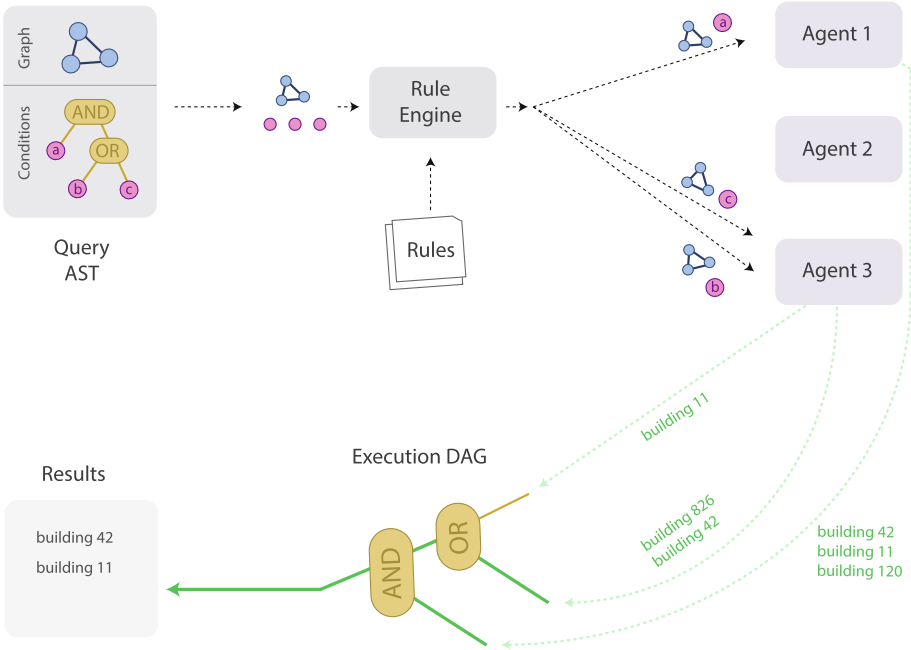


Fig. 15 Query Execution Overview

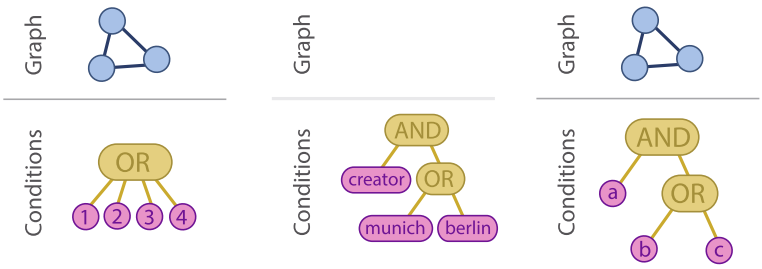


图14 示例q 查询：包含条件的查询、带有指纹的图以及更为复杂的 两者的结合

相关的信息，包括迄今为止匹配的指纹集合以及某一结果的当前得分。合并操作发生在结果沿执行有向无环图（DAG）传递的过程中，或者当某个结果被添加到结果列表且已存在一个完全相同的结果时。例如，若两个具有相同标识的结果同时到达一个“与”节点，则它们会在继续传播之前被合并。数据包支持基于内容的合并操作，例如，将相关的得分值合并为一个新的综合得分。

查询中的每一条件均具有两个可选属性：“固定值”和“权重”，客户端可在发送请求时将其分别设置为任意有理数。若某一结果被某一条件匹配，则该条件的这两个数值将被累加，并计入该结果的总得分。“权重”会与由代理计算得出的介于0至1之间的匹配质量评分相乘。因此，客户端可以利用“固定值”来指示在不考虑质量的情况下确定条件的重要性，而“权重”部分则用于调整质量评分的相对重要性。形式上，对于某一结果x，其综合排名（归一化前）目前按如下公式计算：

$$rank(x) = \sum_{i=0}^{n} \chi_i(x) \cdot \left(f_i + w_i \cdot s_i(x)\right)$$

其中，n 为条件的数量，$\chi_i(x)$ 为第 i 个条件的指示函数，f_i 和 w_i 分别为作为查询组成部分定义的“固定”值和“权重”值，$s_i(x)$ 则是由代理计算得到的得分。

作为数据过滤器的代理

智能体在内部访问数据库中所有建筑的完整集合，并从中生成一个子集。与其让智能体直接作用于底层数据库集合，不如使其作为中间层的过滤器来发挥作用。

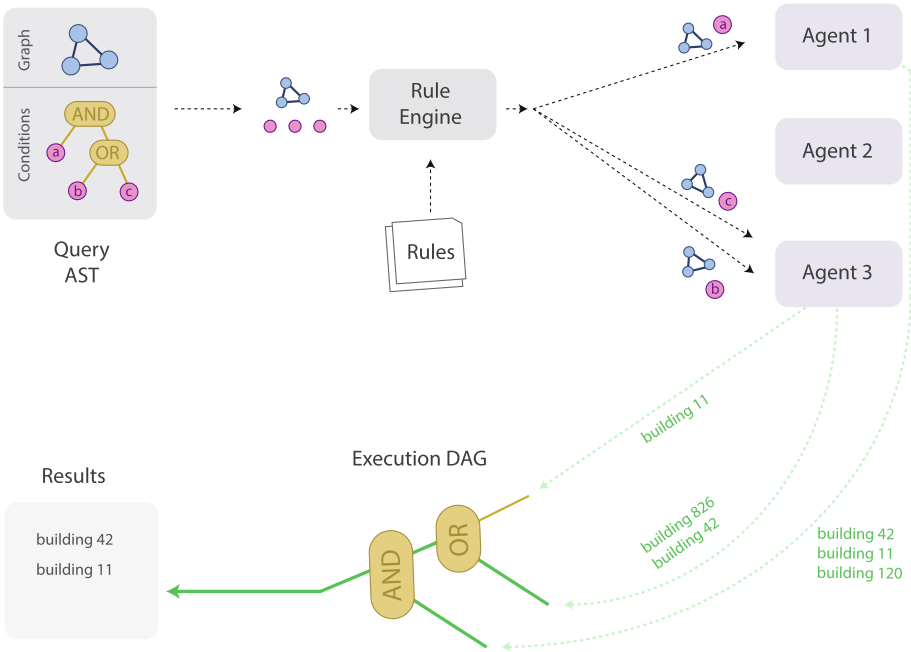


图15 查询执行概览

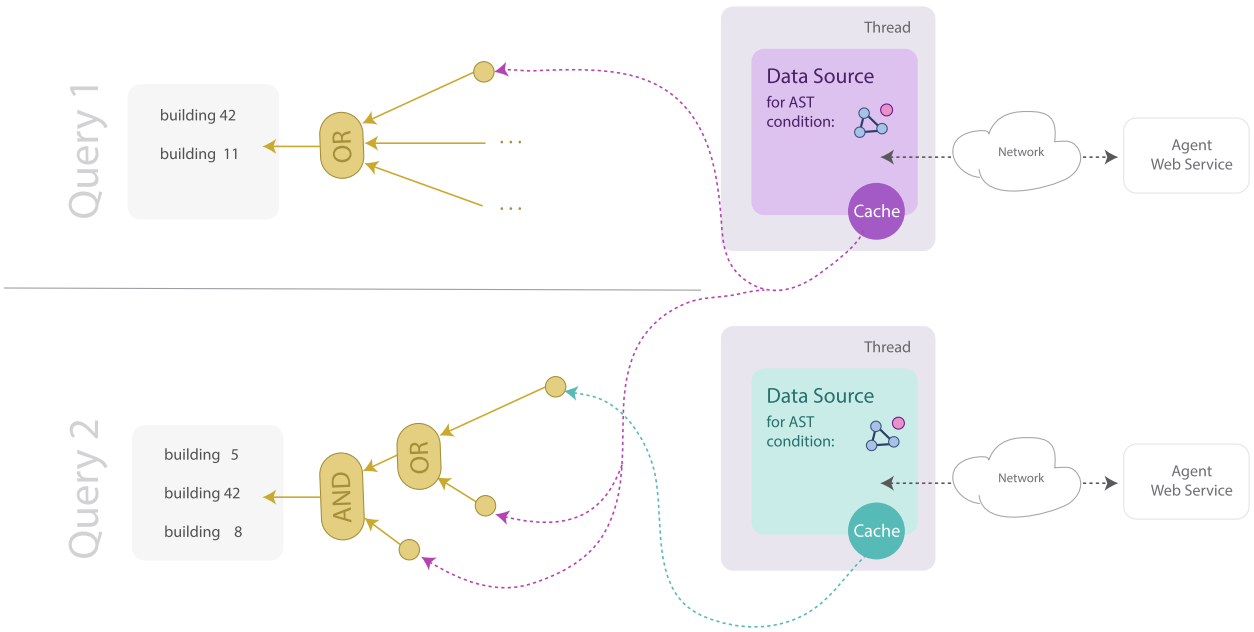


Fig. 16 Data sources are instantiations of agents that communicate with the agent microservices, can be shared across queries and push result candidates through the DAG

sets. Filters have two advantages: First, since intermediate input sets are likely to be smaller than the full underlying set, filters only have to operate on the smaller set and are likely to be faster. Second, filters could potentially depend on the structure of their input when used sequentially. For example, a fingerprint implemented using a filter might assume that its input set already has certain properties, due to being produced by another fingerprint. This can simplify the implementation of an agent. Since the data has to come from somewhere, some agents will always be data sources. Another limitation of filters is that the input set needs to be copied over the network into the agent. Finally, some agents cannot benefit from processing an input set, e.g. if they operate on pre-computed sets of the full database.

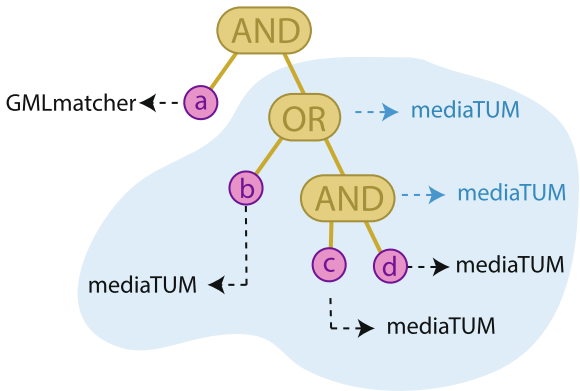


Fig. 17 Subtrees can be executed by a single microservice

Caching and query equality

Coordinator queries are dynamic and updates can happen in both directions: Clients frequently adapt queries in response to user input and result lists change during the execution of long-running queries. To simplify the interface and the clients, clients will always issue new queries. To achieve acceptable performance, the server needs to be able to determine which parts of previous queries are semantically equal and can therefore be reused. We support a query cache, which shares the result list of an existing query and only attaches a new change event listener. In order to determine if a query can be reused, the new query is transformed as usual and the transformed AST is compared to the transformed AST of previous queries. If the ASTs are equal, the results must also be equal and the result list can be reused.

Usually queries are not completely equal: Some parts change due to user interaction with the query sketch. Queries issued by different users will almost certainly be different, especially if they contain a floorplan graph. It is therefore important to also cache intermediate results that are produced by agents. If a matching data source is found, it is reused instead of creating a new one. The most difficult case is dealing with fingerprints. Fingerprints are a special type of condition and they are recursively checked for equality, just like normal conditions. Usually they have

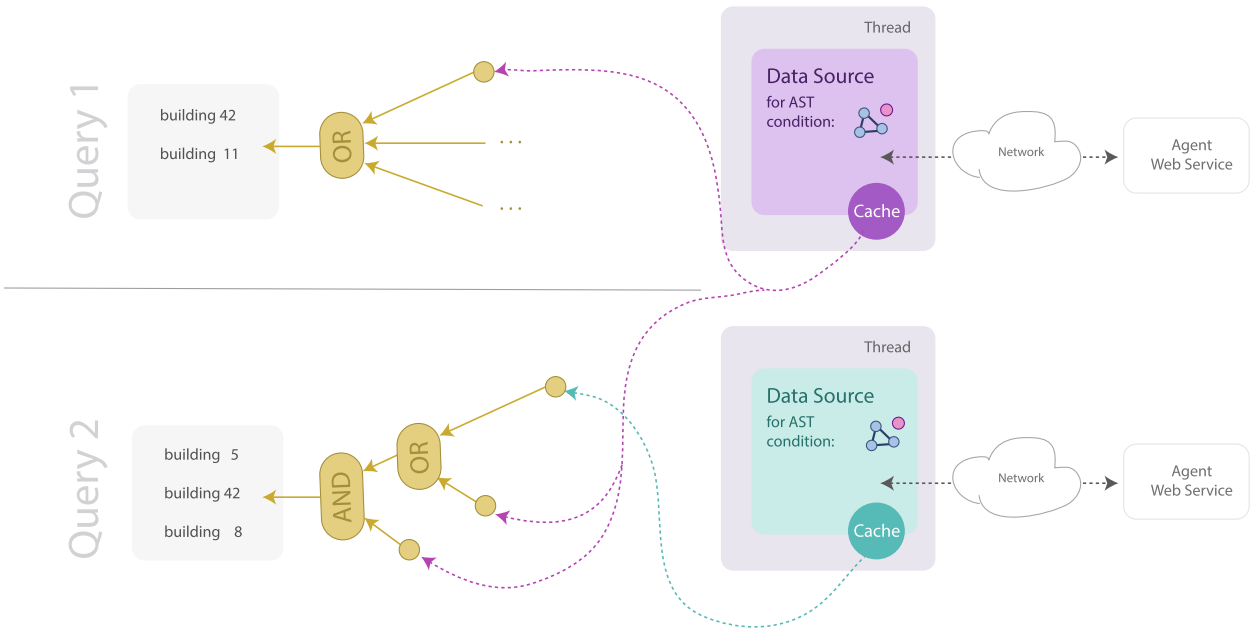


图16 数据源是与代理微服务进行通信的代理实例，可在不同查询间共享，并通过有向无环图（DAG）推送结果候选集。

集合。过滤器具有两大优势：其一，由于中间输入集合通常小于完整的底层集合，过滤器只需在其较小的集合上进行操作，因此往往更为高效；其二，当过滤器被顺序使用时，其行为可能依赖于输入数据的结构。例如，基于过滤器实现的指纹算法可能会假设其输入集合已因前序指纹的处理而具备某些特定性质，这有助于简化代理的实现。鉴于数据必然源自某处，某些代理始终扮演着数据源的角色。此外，过滤器的另一局限在于，输入集合需通过网络复制至代理端。最后，部分代理无法从处理中获益。

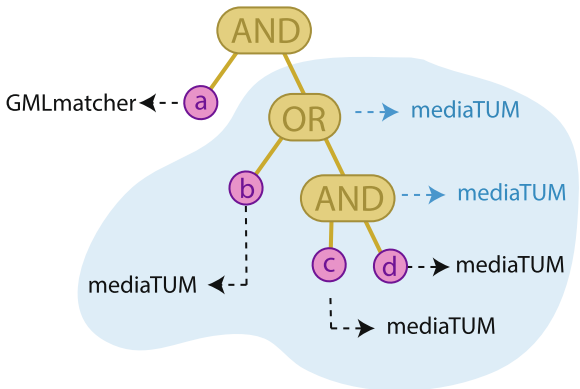


图17 子树可由单个微服务执行

输入集合，例如，当它们作用于预先计算好的完整数据库的集合时。

缓存与查询等价性

协调器查询具有动态性，更新可能在两个方向上发生：客户端会根据用户输入频繁调整查询，而结果列表在长时间运行的查询执行过程中也会发生变化。为简化接口并减轻客户端负担，客户端始终发出新的查询。为了达到可接受的性能，服务器需要能够识别出先前查询中哪些部分在语义上等价，从而实现复用。我们提供了一个查询缓存机制，该机制共享已有查询的结果列表，并仅附加一个新的变更事件监听器。为判断某查询是否可复用，新查询会按常规进行转换，并将其转换后的抽象语法树（AST）与先前查询的转换后AST进行比较。若两者的AST相等，则其结果必然相同，因而可以复用该结果列表。

通常，查询之间并不完全相等：部分查询内容会因用户对查询草图的交互而发生变化。不同用户发出的查询几乎必然存在差异，尤其是在查询中包含楼层平面图的情况下。因此，缓存由各代理生成的中间结果也至关重要。如果找到匹配的数据源，则直接复用该数据源，而非重新创建一个新的。最为复杂的情形是处理指纹条件。指纹条件是一种特殊的条件类型，其相等性判断采用与普通条件相同的递归方式。通常，它们具有

no or few parameters, so the condition node in the AST will often be syntactically equal for fingerprints of the same type. However, fingerprints additionally always refer to a floorplan graph that was passed as part of the query and this graph must match, too. Moreover, the way it must match depends on the type of fingerprint. We implement the graph equality check by defining a pair of helper functions for each type of fingerprint that compute a hash code and provide a function to check equality of two AGraphML graphs respectively. In the example in Fig. 18, the intermediate result set computed by the 'room-count' fingerprint can be reused if the new graph has the same number of rooms. We assume that there is an agent that supports the 'room-count' and 'room-area' fingerprints and can execute them together.

In practice, the new subtree is not compared to all cached subtrees, but instead a hash code is computed and the AST subtree nodes and the graph equivalence helper functions contribute to this hash. The candidates with the same hash are then compared using the above procedure. Although the equality helper functions can be implemented by simply requiring syntactical equivalence of the compared graphs, such a strict comparison will reject many semantically equal graphs. Usually each fingerprint should have its own pair of functions that can perform better. Formally, each fingerprint must provide two functions for hashing and equality, respectively:

$$h_f(g) : G \to \mathbb{Z}$$
$$eq_f\left(g_1, g_2\right) : (G, G) \to \{\text{true}, \text{false}\}$$

where G is the set of all AGraphML documents. For correctness we require: if $eq_f(g_1, g_2)$ then the corresponding fingerprints f_1 and f_2 must produce the same results. This is the only requirement, but for optimal sensitivity fingerprints that always produce the same result sets should also satisfy $eq_f(g_1, g_2) = true$. Similarly for good sensitivity we want: $eq_f(g_1, g_2) \implies h_f(g_1) = h_f(g_2)$. Finally, the usual considerations for good hashing performance apply.

Visualizing results

Starting from the work processes in early design stages and keyword-based search, several methods and tools were developed, including design methods and problem solving strategies. The definition of search queries and the possibility to feed the results back into the design process was a primary concern and was investigated for each of the stages of the design process separately. The project takes a case-based reasoning approach with a special focus on the 'retrieve' step from the CBR cycle by Aamodt and Plaza. Since early architectural design stages focus on the description of the spatial situation primarily through rough sketches, graphical visualizations and drawings, the developed methods and tools also focused on visualizations, e.g. bubble diagrams and sketches. Additionally, the solution had to support certain workflows and interactions:

- Narrowing down the design and refining the graphical queries.
- Definition of multiple levels of abstraction, which include geometrical, topological and alphanumerical data.
- Supporting a flexible work process, which typically involves switching between multiple levels of abstraction when expressing queries. Additionally, users need to be able to start at a suitable point, depending on the stage of the design process.
- Precise specification through logical expressions, comparisons and set operations.
- Prioritizing the derived fingerprints.
- Deriving sequential and parallel search strategies.
- Support of a query history.

Given these goals, requirements and criteria were derived for the formulation of spacial configuration queries. This was done by reviewing appropriate paradigms for human-computer interaction, especially in terms of drawing and sketching and investigating criteria for the

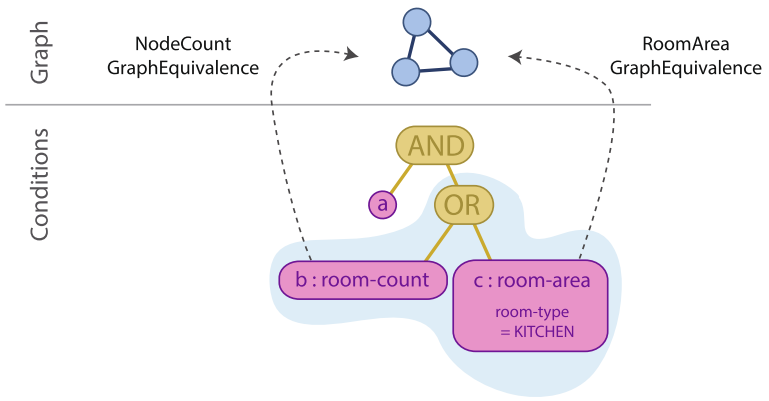


Fig. 18 Caching: Example with two fingerprints

由于参数数量较少或几乎没有参数，因此在抽象语法树（AST）中，同类型指纹的条件节点在语法上往往相等。然而，指纹还始终引用作为查询的一部分传递的楼层平面图，该图也必须匹配。此外，其匹配方式取决于指纹的类型。我们通过为每种指纹类型定义一对辅助函数来实现图的相等性检查：一个用于计算哈希码，另一个用于判断两个 AGraphML 图是否相等。在图 18 的示例中，如果新图具有相同数量的房间，则“房间数”指纹所计算的中间结果集可以被复用。我们假设存在一个支持“房间数”和“房间面积”指纹并能将二者协同执行的代理。

在实践中，新子树并不会与所有已缓存的子树逐一比较，而是先计算其哈希码，AST子树节点以及图等价性辅助函数共同参与该哈希值的生成。随后，具有相同哈希值的候选子树再按照上述流程进行比较。尽管可以通过简单地要求待比较图的语法等价性来实现等价性辅助函数，但这种严格的比较方式会拒绝许多语义上等价的图。通常，每种指纹都应配备一对能够提供更优性能的函数。形式上，每种指纹必须分别提供用于哈希计算和等价性判断的两个函数：

$$h_f(g) : G \to \mathbb{Z}$$
$$eq_f\left(g_1, g_2\right) : (G, G) \to \{\text{true}, \text{false}\}$$

其中，G 表示所有 AGraphML 文档的集合。为保证正确性，我们要求：若 $\mathrm{eq}_f(g_1, g_2)$ 成立，则相应的指纹 f_1 和 f_2 必须产生相同的结果。这是唯一的约束条件；然而，为了获得最佳的敏感性，那些始终产生相同结果集的指纹还应满足 $\mathrm{eq}_f(g_1, g_2) =$ 为真。类似地，为确保良好的敏感性，我们希望：$\mathrm{eq}_f(g_1, g_2) \implies \mathrm{h}_f(g_1) = \mathrm{h}_f(g_2)$。最后，常规的哈希性能考量同样适用。

结果可视化

从设计早期阶段的工作流程以及基于关键词的检索出发，开发了多种方法与工具，其中包括设计方法和问题求解策略。搜索查询的定义及其结果回流至设计过程的可能性是首要关注点，并针对设计过程的各个阶段分别进行了研究。本项目采用基于案例的推理方法，尤其侧重于Aamodt与Plaza提出的CBR循环中的“检索”环节。鉴于建筑方案设计的早期阶段主要通过粗略草图、图形化表达及图纸来描述空间布局，所开发的方法与工具亦以可视化手段为核心，例如气泡图与草图等。此外，该解决方案还需支持特定的工作流程与交互模式：

- 缩小设计范围并优化图形查询。
- 多级抽象的定义，包括几何、拓扑和字母数字数据。
- 支持灵活的工作流程，该流程通常在表达查询时需要在多个抽象层次之间切换。此外，用户还需能够根据设计过程所处的阶段，从合适的起点开始操作。
- 通过逻辑表达式、比较运算和集合运算实现精确的规格定义。
- 优先处理派生的指纹特征。
- 推导顺序与并行搜索策略。
- 支持查询历史记录。

鉴于上述目标，针对空间构型查询的构建，我们推导出了相应的需求与评价标准。这一过程通过梳理人机交互领域的相关范式，尤其是绘图与草图绘制方面的范式，并考察相关评价标准而得以实现。

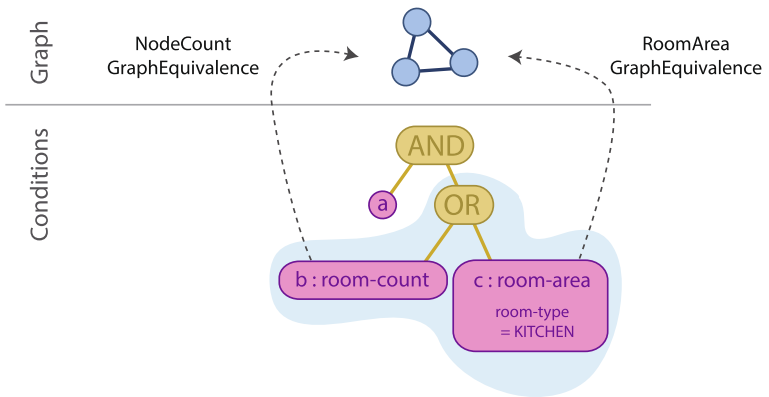


图18 缓存：包含两个指纹的示例

user-oriented design of the user interfaces. An important challenge when developing user interface concepts are the individual, cognitive processes of users. The software has to provide users with a way to express their mental model and find comparable visualizations of the search results. As part of the project, ten prototype frontends were developed to elicit the requirements for the user interface concept and in particular to evaluate possible input methods and devices:

- text-based desktop applications: ar:searchbox (Fig. 6 (Langenhan et al. 2012)) and the mobile application ar:searchbox.app (Langenhan et al. 2013)
- pen-based applications (Figs. 8, 9, 10, 11): a.scatch (Langenhan and Petzold 2015), touchTect, ar:searchDroid, cormorant (Langenhan and Petzold 2016),
- geometrical modeling applications (Figs. 7, 12): ar:searchPad (Langenhan and Petzold 2016), metisWebUI (Bayer et al. 2015), flamingo (Thurow et al. 2016)

These evaluations resulted in multiple requirements for user interaction and user interface metaphors like:

- Interaction and input metaphor based on the desired information density, e.g. bubble diagrams and room volume
- Browsing and using search results as queries
- Suggestion of queries (trends, similar queries)
- User-specific weighting of search criteria

To enable the visualization, exploration and comparison of the results, visualization methods from architecture, visual communication, information and data visualization and interactive data analysis were considered. Understandability and readability were a major concern. Based on the particular fingerprints, the degree of abstraction of the input, and a manual review of the results, candidates like line graphs, scatter plots, and more interactive approaches like coordinated and multiple views, linking and brushing were analyzed using standard tools including Orange, Rapidminer, VisTrails and Aperture Tiles.

Throughout the design process, the spatial models, rooms, and their interrelationships become increasingly concrete in geometrical and topological terms. Which interactions are reasonable and required, depends on the use case and on the user. The topology of the rooms can be formalized as graphs and visualized using edge and vertex diagrams. The implemented system uses such diagrams, since they highlight the spacial relationships between rooms.

When implementing the fingerprint graph equivalence helper functions, it is worth paying particular attention to the common case where a user makes changes to the floorplan graph interactively on a client, which creates new queries after each change. Reusing computed data source sets in this case is very important to achieve good performance:

- *Identifying, adding and deleting nodes.* Some fingerprints need to identify nodes. This can't be done using a graph matching algorithm or graph canonicalization, since such an approach would defeat the purpose of caching. Instead simple heuristics need to be used.
- *Attributes are added, changed or removed for nodes or edges.* Such a change leads to cache misses if trivial implementations like the syntactical equality functions are used. Implementations can easily avoid this by providing a custom graph equivalence implementation and only comparing the attributes that can possibly affect the fingerprint.
- *Edges are temporarily removed by the same client and restored later.* If edges need to be identified on the server, this should not be done using their id, but using the ids of the connected nodes.
- *Nodes are temporarily removed by the same client and restored later.* Clients should try to pick the same node id for nodes that are likely to be the same as a node that previously existed. This is obviously the case for nodes that are restored via an undo/redo feature.

Sorted result list and change events

Packets eventually reach the root node of the execution DAG where they are inserted into an observable result list. The web service layer registers an observer for each query and translates the insert, remove, move, update, and error/warning events into an XML stream that is transferred to the client over HTTP one element at a time. Clients apply each event to a local list to synchronize it.

Clients

Clients can use the results to either show a global overview that does not visualize individual floor plans, but summarizes the resulting data with respect to the individual fingerprints or visualize each result separately. The visualization options range from simple bar and pie charts to complex mappings between query and result and the clustering of result sets. These include an overview of the found floor plans, the fingerprint-based mapping between rooms in the query and a selected result and the data analysis of the result set based on clustering. For example, the metis Web UI displays an overview of the results in the right column of the user interface by showing the floorplan structures from the CMS mediatum, sorted by similarity. Selecting such a floorplan produces

面向用户的设计方法应用于用户界面。在开发用户界面概念时，一个重要的挑战在于用户的个体认知过程。软件必须为用户提供一种表达其心智模型的途径，并使其能够找到与之对应的搜索结果可视化呈现方式。作为项目的一部分，我们开发了十个原型前端，以获取用户界面概念的需求，并重点评估可能的输入方式与输入设备：

- 基于文本的桌面应用程序：ar:searchbox（图6，Langenhan 等，2012）以及移动应用程序 ar:searchbox.app（Langenhan 等，2013）
- 基于笔触的应用程序（图8、9、10、11）：a.scatch（Langenhan和Petzold，2015）、touchTect、ar:searchDroid、cormorant（Langenhan和Petzold，2016），
- 几何建模应用（图7、图12）：ar:searchPad（Langenhan 和 Petzold，2016）、metisWebUI（Bayer 等，2015）、flamingo（Thurow 等，2016）

这些评估产生了多项关于用户交互及用户界面隐喻方面的需求，例如：

- 基于期望信息密度的交互与输入隐喻，例如气泡图和房间体积
- 浏览并将搜索结果用作查询
- 查询建议（趋势、相似查询）
- 用户特定的搜索条件权重设置

为实现结果的可视化、探索与比较，综合考虑了建筑学、视觉传达、信息与数据可视化以及交互式数据分析领域的可视化方法。可理解性和可读性是首要关注点。基于特定的特征指纹、输入数据的抽象程度，并结合对结果的手动审查，采用包括Orange、RapidMiner、VisTrails和Aperture Tiles在内的标准工具，对折线图、散点图以及更具交互性的协同视图、多视图、联动与刷选等方案进行了分析。

在整个设计过程中，空间模型、房间及其相互关系在几何与拓扑层面逐渐趋于具体化。哪些交互是合理且必要的，取决于用例和用户需求。房间的拓扑结构可以形式化为图，并通过边与顶点的示意图进行可视化。所实现的系统采用了此类图示，因为它们能够清晰地展现房间之间的空间关系。

在实现指纹图等价性辅助函数时，值得特别关注。在用户通过客户端交互式地修改平面布局图的常见场景中，每次修改都会产生新的查询。在这种情况下，重用已计算的数据源集合对于实现良好的性能至关重要：

- 节点的识别、添加与删除。某些指纹需要对节点进行识别。这一过程无法通过图匹配算法或图规范化来实现，因为此类方法会违背缓存设计的初衷。因此，必须采用简单的启发式策略。
- 节点或边的属性会被添加、修改或删除。若采用诸如基于语法等价性比较的简单实现，此类变更将导致缓存未命中。通过提供自定义的图等价性判定方法，并仅比较那些可能影响指纹的属性，即可轻松避免这一问题。
- 边由同一客户端暂时移除，并在稍后恢复。如果需要在服务器端识别边，则不应使用其ID，而应使用与其相连节点的ID。
- 节点由同一客户端暂时移除，并在稍后恢复。对于那些很可能与先前存在的节点相同的节点，客户端应尽量为其分配相同的节点标识符。这在通过撤销/重做功能恢复的节点中尤为明显。

排序后的结果列表与变更事件

数据包最终到达执行有向无环图的根节点，并被插入到可观测的结果列表中。Web服务层为每条查询注册一个观察者，并将插入、删除、移动、更新以及错误/警告事件转换为XML流，通过HTTP协议逐个元素地传输至客户端。客户端将每个事件应用于本地列表，以实现同步。

客户

客户端可以利用这些结果，既可展示全局概览——该概览不呈现具体的楼层平面图，而是基于各指纹对结果数据进行汇总；也可分别可视化每个结果。可视化选项从简单的柱状图和饼图，扩展至查询与结果之间的复杂映射以及结果集的聚类分析。具体包括：已发现楼层平面图的概览、基于指纹的查询房间与选定结果之间的映射，以及基于聚类的结果集数据分析。例如，metis Web界面在用户界面的右侧列中显示结果概览，以相似度为依据，按顺序排列来自 CMS mediatum 的楼层平面结构。选择某一楼层平面后，系统将生成

a mapping view, making it possible to evaluate the quality of this result by comparing the room configurations. The prototype client applications a.scatch, TouchTect, ar:searchdroid and cormorant accept input from freehand drawings which are created using a digital pen. The quality of the different visualizations and interactions was evaluated in a user study (Bayer et al. 2015). This initial proof of concept indicated that while most participants liked the prototypes, paper and pen were still the preferred method.

The required properties of information systems for the support of early design stages were elicited through a heuristic evaluation (see Fig. 19). The importance-satisfaction diagram shows, which properties a design tool should have in the early design stages and how well these properties can be realized in the described search scenario. Seven architects were asked to specific the importance of certain properties for searching based on freehand drawings from their point of view. Afterwards, a second questionnaire was used to assess the degree of implementability of the properties. Of the 24 properties (Fig. 19, see (Langenhan 2017, p. 88) for a complete list of properties), 17 (green area) were rated to be implementable or very implementable and further development was recommended. Two-dimensional visualizations (4.5), drawing of design ideas (4.2) and freehand drawings (3.1) had the highest priority for further development. Only the realization of schematic visualizations (4.1) required improvements (red area) and had to be revised. The aim of future user studies will be an assessment of the property 4.1 in the green area. Properties in the blue area were assigned a low priority for further development. Building information models (BIM) were rated as not important and not practicable and accordingly have the lowest priority.

Conclusions and future work

Our system selects agents using a database of rules when executing a condition in a query. The goal is to pick agents that are computationally efficient or produce good results. The rule-based approach is straightforward to implement and makes it easy to understand and to visualize why a particular decision was taken. A drawback is that it is difficult to determine good decision values for the various

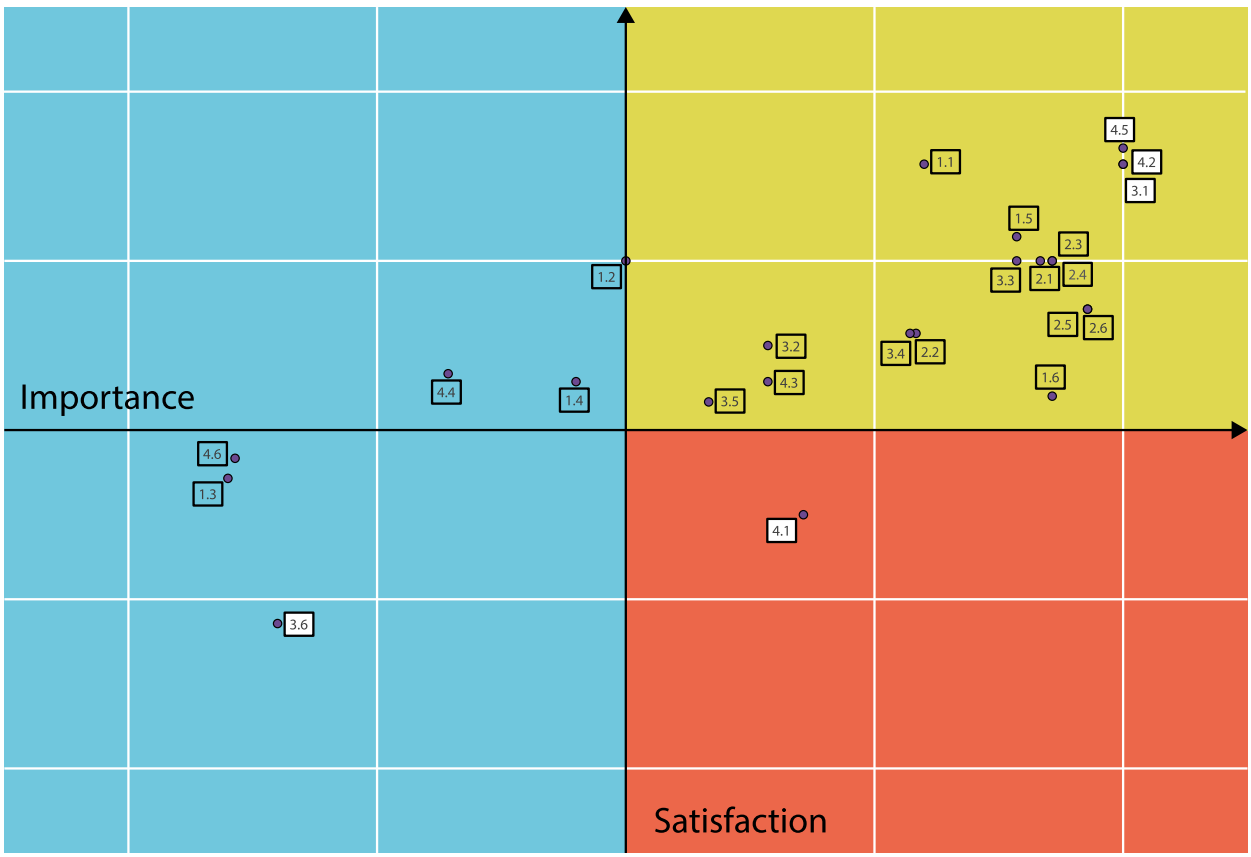


Fig. 19 Importance-Satisfaction diagram

在映射视图中，通过对比房间布局，可以评估该结果的质量。原型客户端应用程序 a.scatch、TouchTect、ar:se archdroid 和 cormorant 均可接受由数字笔绘制的自由手绘输入。不同可视化方案及交互方式的质量已在一项用户研究中得到评估（Bayer 等，2015）。这一初步的概念验证表明，尽管大多数参与者对这些原型表示认可，但纸笔仍是他们首选的输入方式。

通过启发式评估，提炼出了用于支持设计早期阶段的信息系统所需具备的特性（见图19）。重要性—满意度图展示了在设计早期阶段，设计工具应具备哪些特性，以及这些特性在所描述的检索场景中能够得到多大程度的实现。我们邀请了七位建筑师，从其个人视角出发，针对基于手绘草图的检索任务，对各项特性的相对重要性进行评分。随后，又通过第二份问卷评估了这些特性在实际中的可实现程度。在24项特性中（图19；完整列表参见Langenhan，2017，第88页）

在各项属性中，有17项（绿色区域）被评定为可实施或非常可实施，并建议进一步开发。其中，二维可视化（4.5）、设计构思的绘制（4.2）以及手绘草图（3.1）被列为优先级最高的进一步开发对象。唯有示意图的实现（4.1）存在改进需求（红色区域），需予以修订。未来用户研究的目标将是对绿色区域内属性4.1进行评估。蓝色区域内的属性则被赋予较低的进一步开发优先级。建筑信息模型（BIM）被评定为既不重要也不具可操作性，因此其优先级最低。

结论与未来工作

在查询中执行条件时，我们的系统采用基于规则的数据库来选择代理。其目标是选取计算效率高或能产生良好结果的代理。基于规则的方法实现起来较为简单，并且易于理解与可视化，从而清晰地展示为何会做出某一特定决策。其缺点在于，难以确定各类决策参数的合理取值。

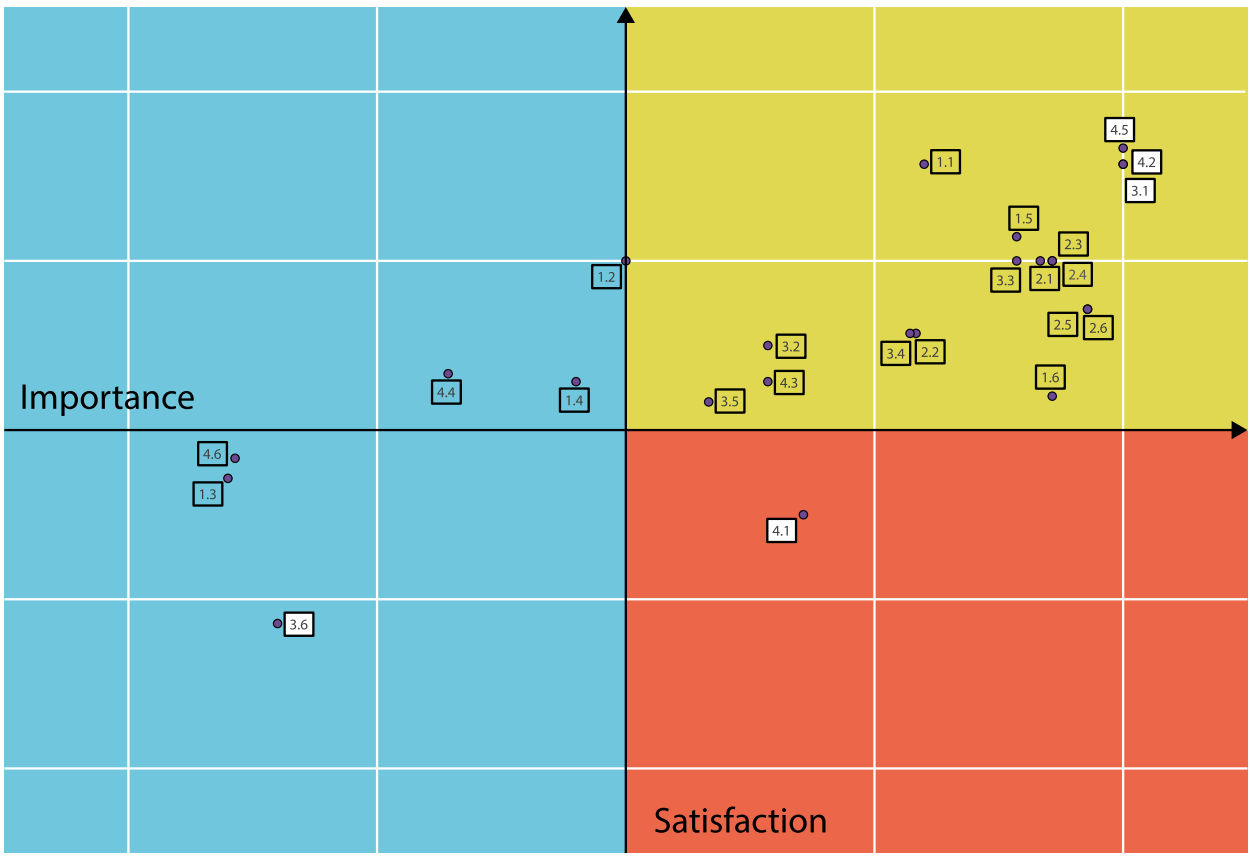


图19 重要性-满意度图

features and that the rule database has to be maintained by hand. Instead of producing the rules by hand, it might be useful to learn them using machine learning. To keep the benefit of an intelligible system with easily traceable decisions, rule learning or decision trees are a good candidate (Flach 2012). A recent overview of rule learning is published in (Fürnkranz and Kliegr 2015). By keeping the intermediate results of executions and tracking the execution time of each agent, this approach could be taken further to learn and incrementally improve the rule set online. Clients could be extended to provide feedback on the quality of results, for example by allowing users to rate them.

Goldschmidt and Smalkov have posed the underlying question of whether visual thinking is derived from mental processes or from preceding visual images (Goldschmidt and Smolkov 2006, p. 549). The history of architecture shows that design tools have influenced how the built environment was made. The influence of tools on the thinking process is, however, hard to measure. Gaenshirt has argued that every software contains a hidden ideology (Gänshirt 2007, p. 193) which conditions every object constructed with them. It might be useful to keep a history of query results and possibly also intermediate sets permanently, for example to support the machine learning case described above or other data mining scenarios. The persisted data needs to be versioned to account for changes in the database and in the involved services.

We have described the design and implementation of the Coordinator, a middleware to support graph-based floorplan retrieval. The system accepts queries in a unified query language and executes them using agents which are selected using a set of rules. Furthermore, queries can be updated and the Coordinator achieves efficient execution using a domain-specific caching framework. Future work includes automatically learning the rules, providing support for online analysis of materialized datasets and improving the prototype implementation.

Endnotes

[1] "Data integration is the problem of providing unified and transparent access to a set of autonomous and heterogeneous sources, in order to allow for expressing queries that could not be supported by the individual data sources alone." (Keim et al. 2010, p. 23)

[2] Semantically equal means that the queries describe the same set of results, but are phrased slightly differently.

Acknowledgements
Not applicable.

Funding
The work presented was supported by the German Research Foundation (DFG) as part of the 'metis' research project.

Availability of data and materials
Not applicable.

Authors' contributions
JR is responsible for the computer-science related parts of the paper. CL and FP are responsible for the architecture-related parts of the paper. All authors read and approved the final manuscript.

Ethics approval and consent to participate
Not applicable.

Consent for publication
Not applicable.

Competing interests
The authors declare that they have no competing interests.

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Author details
[1] Department of Computer Science, Technical University of Munich, Munich, Germany. [2] Department of Architecture, Technical University of Munich, Munich, Germany.

Received: 11 November 2016 Accepted: 25 September 2017
Published online: 24 October 2017

References

Bayer, J, Bukhari, S, Langenhan, C, Eichenberger-Liwicki M, Althoff KD, Petzold, F, Dengel, A (2015). Migrating the classical pen-and-paper based conceptual sketching of architecture plans towards computer tools: Prototype design and evaluation, In *GREC 2015*. Springer International Publishing. doi:10.1007/978-3-319-52159-6_4.

Beck, F, Burch, M, Diehl, S, Weiskopf, D (2014). The State of the Art in Visualizing Dynamic Graphs, In *Proceedings State of the Art Reports (STARs)*. 1605.08485 (pp. 83–103).

Buxton, W (2007). *Sketching User Experience. Getting the Design Right and the Right Design*. San Francisco: M. Kaufmann.

Conte, D, Foggia, P, Sansone, C, Vento, M (2004). Thirty Years of Graph Matching in Pattern Recognition. *International Journal of Pattern Recognition and Artificial Intelligence*, *18*(03), 265–298.

Dengel, A (2012). *Semantische Technologien*, (p. 461). http://link.springer.com/10.1007/978-3-8274-2664-2.

Flach, P (2012). *Machine Learning. The Art and Science of Algorithms that Make Sense of Data vol. 2010*, (p. 396). New York: Cambridge University Press.

Fürnkranz, J, & Kliegr, T (2015). A Brief Overview of Rule Learning. In N Bassiliades, G Gottlob, F Sadri, A Paschke, D Roman (Eds.), *Rule Technologies: Foundations, Tools, and Applications SE - 4*. Lecture Notes in Computer Science. http://link.springer.com/10.1007/978-3-319-21542-6_4, 9202 (pp. 54–69). Berlin: Springer.

Gänshirt, C (2007). Werkzeuge Für Ideen. Einführung Ins Architektonische Entwerfen., Basel. http://www.gbv.de/dms/hebis-darmstadt/toc/187535930.pdf.

Goldschmidt, G, & Smolkov, M (2006). Variances in the impact of visual stimuli on design problem solving performance. *Design Studies*, *27*(5), 549–569.

GraphML (2017). The GraphML File Format. http://graphml.graphdrawing.org/. Accessed 4 May 2011.

Gratzl, S, Lex, A, Gehlenborg, N, Pfister, H, Streit, M (2013). LineUp: Visual analysis of multi-attribute rankings. *IEEE Transactions on Visualization and Computer Graphics*, *19*(12), 2277–2286.

Hanson, J (1998). *Decoding Homes and Houses*. Cambridge; New York: Cambridge University Press.

Keim, D, Kohlhammer, J, Ellis, G, Mansmann, F (2010). *Mastering the information age: Solving problems with visual analytics*. Goslar: Eurographics Association.

Langenhan, C (2017). Data Management in Architecture - Investigating the organisation of design information in IT infrastructures and identifying potential uses in knowledge-based systems. Dissertation. Technical University of Munich. ISBN 978-620-2-32014-6. Online verfügbar: https://mediatum.ub.tum.de/doc/1335437/1335437.pdf.

特征，并且规则库需要手动维护。与其手工构建规则，不如利用机器学习来自动学习这些规则。为了保持系统具有可理解性且决策过程易于追溯的优点，规则学习或决策树不失为一种理想的选择（Flach，2012）。关于规则学习的最新综述见于（Fürnkranz 和 Kliegr，2015）。通过保存执行过程中的中间结果并跟踪各代理的执行时间，这一方法还可进一步扩展，以在线方式学习并逐步优化规则集。客户端可以扩展功能，用于反馈结果的质量，例如允许用户对结果进行评分。

Goldschmidt和Smalkov提出了一个根本性问题：视觉思维究竟源于心理过程，还是源于先前的视觉表象（Goldschmidt和Smolkov，2006，第549页）。建筑史表明，设计工具对建成环境的营造方式产生了重要影响。然而，工具对思维过程的影响却难以量化。Gaenshirt曾指出，每一种软件都蕴含着一种隐性的意识形态（Gänshirt，2007，第193页），而这种意识形态会制约使用该软件所构建的每一个对象。或许有必要长期保存查询结果，甚至中间集合，例如用于支持上述机器学习场景或其他数据挖掘应用。为应对数据库及所涉及服务的变化，这些持久化数据还需进行版本管理。

我们阐述了Coordinator的设计与实现，这是一种用于支持基于图的平面布局检索的中间件。该系统接受以统一查询语言表达的查询，并通过依据一组规则选择的代理来执行这些查询。此外，查询可以被更新，且Coordinator借助面向特定领域的缓存框架实现了高效的查询执行。未来的工作包括自动学习规则、为物化数据集的在线分析提供支持，以及进一步优化原型系统的实现。

尾注

1 “数据集成是指为一组自治且异构的数据源提供统一且透明的访问接口，从而支持表达那些仅靠各个数据源自身无法实现的查询。”（Keim 等，2010，第23页）

2 语义等价是指这些查询虽然表述略有不同，但所描述的结果集是相同的。

致谢：不适用。

资助
本文工作得到了德国研究基金会（DFG）在“metis”研究项目框架下的资助。

数据与材料的可获得性
不适用。

作者贡献
JR负责论文中与计算机科学相关的内容，CL和FP负责论文中与体系结构相关的内容。所有作者均已审阅并批准最终稿。

伦理批准及参与同意
不适用。

出版同意：不适用。

利益冲突
作者声明不存在利益冲突。

出版者注

施普林格·自然对已发表地图中的管辖权主张及机构隶属关系保持中立立场。

作者详情
1德国慕尼黑工业大学计算机科学系，德国慕尼黑。[2]德国慕尼黑工业大学建筑学系，德国慕尼黑。

收稿日期：2016年11月11日 接受日期：2017年9月25日
Published online: 24 October 2017

参考文献

Bayer, J，Bukhari, S，Langenhan, C，Eichenberger-Liwicki M，Althoff K D，Petzold, F，Dengel, A（2015）。将基于传统纸笔的建筑方案概念草图绘制向计算机工具迁移：原型设计与评估，载于 GREC 2015。Springer International Publishing。doi:10.1007/978-3-319-52159-6_4。Beck, F，Burch, M，Diehl, S，Weiskopf, D（2014）。动态图可视化研究现状，载于《研究前沿报告》（STARs）。1605.08485（第83–103页）。Buxton, W（2007）。用户体验草图设计：做对的设计与正确的设计。旧金山：M. Kaufmann。Conte, D，Foggia, P，Sansone, C，Vento, M（2004）。模式识别领域图匹配三十年。国际模式识别与人工智能杂志，18(03)，265–298。Dengel, A（2012）。语义技术。（第461页）。http://link.springer.com/10.1007/978-3-8274-2664-2。Flach, P（2012）。机器学习：数据理解算法的艺术与科学，第2010卷。（第396页）。纽约：剑桥大学出版社。Fürnkranz, J，Kliegr, T（2015）。规则学习简述。载于 N Bassiliades、G Gottlob、F Sadri、A Paschke、D Roman（编），《规则技术：基础、工具与应用》，SE - 4。计算机科学讲义。http://link.springer.com/10.1007/978-3-319-21542-6_4，9202（第54–69页）。柏林：Springer。Gänshirt, C（2007）。创意工具：建筑创作导论。巴塞尔。http://www.gbv.de/dms/hebis-darmstadt/toc/187535930.pdf。Goldschmidt, G，Smolkov, M（2006）。视觉刺激对设计问题解决绩效影响的差异性分析。设计研究，27(5)，549–569。GraphML（2017）。GraphML 文件格式。http://graphml.graphdrawing.org/。访问日期：2011年5月4日。Gratzl, S，Lex, A，Gehlenborg, N，Pfister, H，Streit, M（2013）。LineUp：多属性排序的可视化分析。IEEE 可视化与计算机图形学汇刊，19(12)，2277–2286。Hanson, J（1998）。解读住宅与房屋。剑桥；纽约：剑桥大学出版社。Keim, D，Kohlhammer, J，Ellis, G，Mansmann, F（2010）。驾驭信息时代：以可视化分析解决问题。戈斯拉：欧洲图形学会。Langenhan, C（2017）。建筑中的数据管理——探究IT基础设施中设计信息的组织方式并识别其在知识型系统中的潜在应用。博士论文。慕尼黑工业大学。ISBN 978-620-2-32014-6。在线获取地址：https://mediatum.ub.tum.de/doc/1335437/1335437.pdf。

Langenhan, C, & Petzold, F (2010). The fingerprint of architecture: sketch-based design methods for researching building layouts through the semantic fingerprinting of floor plans. *International electronic scientific-educational journal: Architecture and Modern Information Technologies*, *4*(13), 1–8. http://www.marhi.ru/eng/AMIT/2010/4kvart10/langenhan/abstract.php.

Langenhan, C, & Petzold, F (2015). BEYOND THE BUBBLE – Computer-aided topological analysis and parametric de-sign of room configurations in university education, In *eCAADe 2015 – 33rd Annual Conference 16th–18th September 2015. Vienna*: eCAADe (Education and research in Computer Aided) Architectural Design in Europe) and Faculty of Architecture and Urban Planning, TU Wien.

Langenhan, C, & Petzold, F (2016). Perspectives on architecture: Different abstraction layer of building information imply special working methods and interaction metaphors to support a variety course of action, In *Sigradi, 2016. XX Conference of the Iberoamerican Society of Digital Graphics*.

Langenhan, C, Sahm, A, Seifert, A, Teichert, A, Petzold, F (2013). Mobile application to collect information about architecture to obtain a collective knowledge base, In *ASCAAD 2013*.

Langenhan, C, Seifert, A, Teichert, A, Petzold, F (2012). ar:searchbox: Knowledge management for architecture students, In *eCAADe*: eCAADe (Education and research in Computer Aided) Architectural Design in Europe) and ČVUT, Faculty of Architecture.

Langenhan, C, Weber, M, Liwicki, M, Petzold, F, Dengel, A (2013). Graph-based retrieval of building information models for supporting the early design stages. *Advanced Engineering Informatics*, *27*(4), 413–426.

Liebich, T (1994). *Wissensbasierter Architekturentwurf. Von Den Modellen des Entwurfs zu Einer Intelligenten Computerunterstützung*. Weimar: VDG, Verlag und Datenbank für Geisteswissenschaften.

Richter, K (2010). *Augmenting Designers' Memory. Case Based Reasoning in der Architektur*. Berlin: Logos-Verlag.

Roith, J, Langenhan, C, Petzold, F (2016). Supporting the building design process with graph-based methods using centrally coordinated federated databases, In *ICCCBE*. Osaka.

Roth-Berghofer, TR, & Richter, MM (2008). On explanation. *Künstliche Intelligenz*, *22*(2), 5–7.

Thurow, T, Langenhan, C, Petzold, F (2016). Assisting early architectural planning using a geometry-based graph search, In *eCAADe 2016*.

Vatavu, RD, Anthony, L, Wobbrock, JO (2015). $P Point-Cloud Recognizer. https://depts.washington.edu/aimgroup/proj/dollar/pdollar.html.

Weber, M, Langenhan, C, Roth-Berghofer, T, Liwicki, M, Dengel, A, Petzold, F (2010). a.scatch semantic structure for architectural floor plan retrieval. In I Bichindaritz & S Montani (Eds.), *ICCBR*, 6176 (pp. 2010–6176510524). Berlin: Springer.

Weber, M, Langenhan, C, Roth-Berghofer, T, Liwicki, M, Dengel, A, Petzold, F (2011). Fast subgraph isomorphism detection for graph-based retrieval. *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, *6880 LNAI*, 319–333.

Submit your manuscript to a SpringerOpen journal and benefit from:

- ► Convenient online submission
- ► Rigorous peer review
- ► Open access: articles freely available online
- ► High visibility within the field
- ► Retaining the copyright to your article

Submit your next manuscript at ► springeropen.com

Langenhan, C, & Petzold, F（2010）。建筑的“指纹”：基于草图的设计方法——通过平面图的语义指纹分析研究建筑布局。国际电子科学教育期刊：建筑与现代信息技术，4(13)，1–8。http://www.marhi.ru/eng/AMIT/2010/4kvart10/langenhan/abstract.php。 Langenhan, C, & Petzold, F（2015）。超越“泡沫”——面向高校教学的房间构型计算机辅助拓扑分析与参数化设计。载于 eCAADe 2015——第33届年度会议，2015年9月16日至18日，维也纳：eCAADe（欧洲计算机辅助建筑设计教育与研究）及维也纳工业大学建筑与城市规划学院。 Langenhan, C, & Petzold, F（2016）。建筑学视角：建筑信息的不同抽象层次决定了特定的工作方法与交互隐喻，以支持多样化的行动路径。载于 Sigradi 2016，伊比利亚美洲数字图形学会第二十次大会。 Langenhan, C, Sahm, A, Seifert, A, Teichert, A, Petzold, F（2013）。用于收集建筑信息并构建集体知识库的移动应用。载于 ASCAAD 2013。 Langenhan, C, Seifert, A, Teichert, A, Petzold, F（2012）。ar:searchbox：面向建筑学学生的知识管理。载于 eCAADe：欧洲计算机辅助建筑设计教育与研究，以及捷克技术大学建筑学院。 Langenhan, C, Weber, M, Liwicki, M, Petzold, F, Dengel, A（2013）。基于图模型的建筑信息模型检索，用于支持设计前期阶段。《先进工程信息学》，27(4)，413–426。 Liebich, T（1994）。基于知识的建筑设计：从设计模型到智能计算机辅助。魏玛：VDG人文科学出版社与数据库。 Richter, K（2010）。增强设计师的记忆：建筑领域的案例推理。柏林：Logos出版社。 Roith, J, Langenhan, C, Petzold, F（2016）。利用集中协调的联邦数据库与基于图的方法支持建筑设计过程。载于 ICCCBE，大阪。 Roth-Berghofer, TR, & Richter, MM（2008）。论解释。《人工智能》，22(2)，5–7。 Thurow, T, Langenhan, C, Petzold, F（2016）。基于几何的图搜索辅助早期建筑规划。载于 eCAADe 2016。 Vatavu, RD, Anthony, L, Wobbrock, JO（2015）。$P 点云识别器。https://depts.washington.edu/aimgroup/proj/dollar/pdollar.html。 Weber, M, Langenhan, C, Roth-Berghofer, T, Liwicki, M, Dengel, A, Petzold, F（2010）。a.scatch语义结构用于建筑平面图检索。载于 I Bichindaritz & S Montani（编），ICCBR，6176（页码2010–6176510524）。柏林：施普林格出版社。 Weber, M, Langenhan, C, Roth-Berghofer, T, Liwicki, M, Dengel, A, Petzold, F（2011）。面向基于图检索的快速子图同构检测。《计算机科学讲义》（含人工智能讲义与生物信息学讲义子系列），6880 LNAI，319–333。

Submit your manuscript to a SpringerOpen journal and benefit from:

- ► Convenient online submission
- ► Rigorous peer review
- ► Open access: articles freely available online
- ► High visibility within the field
- ► Retaining the copyright to your article

Submit your next manuscript at ► springeropen.com